

Amélioration de l'observabilité d'une application Web par l'intégration de la télémétrie frontend

RAPPORT TECHNIQUE : APPLICATION DE LA DEMARCHE
SCIENTIFIQUE
MIONNET, JONAS

Table des matières

Résumé.....	1
Abstract.....	2
1 Introduction	3
1.1 Contexte général des applications web modernes.....	3
1.2 Importance de l'observabilité dans les systèmes informatiques.....	3
1.3 Présentation du sujet.....	3
1.4 Organisation du rapport	4
2 Contexte et périmètre de l'étude.....	4
2.1 Présentation de l'entreprise et du projet	4
2.2 Description de l'application étudiée	4
2.3 Architecture générale du système.....	4
2.4 Périmètre de l'étude	5
2.5 Contraintes techniques et organisationnelles	5
3 Problématique et objectifs.....	5
3.1 Analyse de la situation actuelle	5
3.2 Limites de l'observabilité actuelle	5
3.3 Problématique de l'étude	6
3.4 Objectifs du projet.....	6
3.5 Enjeux techniques et métier	6
4 Analyse de l'existant.....	6
4.1 Observabilité actuelle du backend	6
4.1.1 Instrumentation OpenTelemetry.....	7
4.1.2 Collecte, export et visualisation des données.....	8
4.1.3 Propagation des traces.....	8
4.2 Données actuellement collectées.....	8
4.3 Apports de la solution actuelle.....	9
4.4 Limitations de la solution actuelle.....	9
4.5 Synthèse	10
5 Etat de l'art : observabilité des applications	10
5.1 Définition de l'observabilité.....	10
5.2 Logs, métriques et traces.....	11
5.3 Observabilités dans les architectures distribuées	11
5.4 Présentation d'OpenTelemetry.....	12
5.5 Panorama des solutions d'observabilité frontend.....	12
5.6 Observabilité Frontend.....	13

5.6.1	Deux approches complémentaires : Synthetic Monitoring et Real User Monitoring	13
5.6.2	Les Web Vitals : un standard pour mesurer la performance perçue.....	13
5.6.3	Les catégories de données collectées côté frontend	14
5.6.4	La corrélation session / trace : un enjeu central	14
5.6.5	Les contraintes spécifiques au navigateur.....	14
6	Etudes de solutions possibles	15
6.1	Solutions de télémétrie frontend existantes	15
6.2	Instrumentation du navigateur.....	16
6.3	Corrélation frontend / backend.....	17
6.4	Comparaison des solutions.....	17
7	Choix de la solution.....	18
7.1	Critères de sélection	18
7.2	Analyse des solutions selon les critères	19
7.3	Solution retenue	20
7.4	Architecture de la solution proposée	21
8	Mise en œuvre et méthodologie de validation.....	22
8.1	Environnement technique	22
8.2	Résolution du problème d'export depuis le navigateur	23
8.2.1	Contrainte fondamentale : le navigateur ne supporte pas gRPC	23
8.2.2	Ouverture d'un second endpoint OTLP dans Aspire	23
8.2.3	Mise en place d'un proxy Vite pour relayer les traces	23
8.3	Instrumentation du frontend.....	24
8.3.1	Parcours pilote retenu	24
8.3.2	Installation des packages.....	24
8.3.3	Initialisation du SDK.....	25
8.3.4	Restriction de la propagation du traceparent	25
8.3.5	Instrumentation automatique.....	26
8.3.6	Instrumentation manuelle	26
8.4	Collecte des données de télémétrie	27
8.4.1	Chaîne d'export des traces	27
8.4.2	Corrélation avec le backend	27
8.5	Méthodologie de validation	27
8.5.1	Validation de l'objectif 1 : Collecte d'événements frontend	27
8.5.2	Validation de l'objectif 2 : Corrélation frontend/backend.....	28
8.5.3	Validation de l'objectif 3 : Absence de données sensibles.....	28
8.5.4	Validation de l'objectif 4 : Absence de dégradation des performances.....	28

8.6	Mise en place des tests.....	28
8.6.1	Scénario 1 : Vérification de la collecte des spans frontend	28
8.6.2	Scénario 2 : Vérification de la corrélation frontend/backend	29
8.6.3	Scénario 3 : Vérification de l'absence de données sensibles	29
8.6.4	Scénario 4 : Vérification de l'impact sur les performances	29
9	Résultats et analyse.....	29
9.1	Données collectées.....	29
9.2	Corrélation frontend / backend	30
9.3	Vérification de la confidentialité.....	30
9.4	Analyse des performances.....	31
9.5	Limites de la solution	31
10	Conclusion et perspectives	31
10.1	Synthèses des résultats.....	31
10.2	Améliorations possibles.....	32
10.3	Perspectives.....	33
11	Annexes.....	34
12	Bibliographie	43

Listes des tableaux

Tableau 1 : Comparaison des solutions étudiées	18
Tableau 2 : Environnement technique de mise en œuvre	22
Tableau 3: Métriques frontend définies dans otel.ts	25
Tableau 4 : Spans manuels définis dans le parcours pilote.....	26

Résumé

Dans un contexte où les applications web reposent sur des architectures distribuées composées de plusieurs services interconnectés, l'observabilité des systèmes informatiques constitue un enjeu majeur pour les équipes de développement et d'exploitation. Elle permet de mieux comprendre le comportement d'une application en production, d'identifier rapidement les anomalies (lenteurs, erreurs) et d'améliorer les performances globales du système.

Dans le cadre d'un projet réalisé au sein d'une application web utilisée dans le secteur de l'assurance, une solution d'observabilité basée sur OpenTelemetry [1] est actuellement déployée côté backend. Cette solution permet de collecter certaines données techniques telles que les temps de réponse des API, les erreurs serveur ou encore différentes traces de traitement. Toutefois, l'observabilité reste incomplète : aucune télémétrie n'est mise en place côté frontend, ce qui empêche de disposer d'une vision claire des actions réalisées par les utilisateurs dans l'interface web.

Cette absence d'informations rend plus complexe l'analyse des performances globales de l'application et limite la compréhension de l'expérience utilisateur. Il devient notamment difficile d'établir un lien entre une action utilisateur (par exemple un clic ou un changement d'écran) et les traitements réalisés côté serveur.

L'objectif de cette étude est d'analyser comment l'intégration d'une solution de télémétrie frontend peut améliorer l'observabilité globale de l'application. L'étude s'appuie sur l'analyse de l'existant, la réalisation d'un état de l'art sur les solutions modernes de télémétrie et l'examen de différentes approches d'instrumentation frontend. Le périmètre de l'étude concerne l'instrumentation d'un parcours utilisateur représentatif au sein du frontend de l'application, en veillant à respecter les contraintes techniques et organisationnelles du projet (performances du navigateur, confidentialité, normes internes).

Une solution basée sur une approche hybride OpenTelemetry [1], combinant instrumentation automatique des appels réseau et instrumentation manuelle des étapes fonctionnelles, a été mise en œuvre sur un parcours utilisateur pilote. Cette mise en œuvre a nécessité de résoudre plusieurs contraintes techniques propres à l'environnement navigateur, notamment l'incompatibilité avec le protocole gRPC, le format d'export imposé par Aspire, la gestion des politiques CORS via un proxy Vite et la restriction de la propagation du contexte de trace aux seules requêtes internes.

Les résultats obtenus confirment l'amélioration effective de la visibilité sur le comportement de l'application. La corrélation entre les actions utilisateur et les traitements backend fonctionne de bout en bout, les spans frontend et backend partageant le même identifiant de trace dans le dashboard Aspire. L'instrumentation respecte les contraintes de confidentialité et n'introduit pas de dégradation perceptible des performances. Cette étude démontre ainsi qu'il est possible, dans le cadre d'un projet existant et sans modification de l'architecture backend, d'étendre la chaîne d'observabilité jusqu'au navigateur et d'obtenir une vision continue du comportement de l'application, du clic utilisateur jusqu'aux traitements distribués dans les microservices.

Abstract

In modern software systems, web applications are often built on distributed architectures composed of multiple interconnected services. In such environments, application observability has become a critical capability for development and operations teams. Observability enables a better understanding of system behavior in production, facilitates faster detection of anomalies, and contributes to improving overall application performance.

Within the context of a web application used in the insurance sector, an observability solution based on OpenTelemetry is currently implemented on the backend side. This solution allows the collection of technical data such as API response times, server errors, and processing traces. However, the current observability remains incomplete because no telemetry is collected from the frontend layer. As a result, technical teams have limited visibility into user interactions occurring within the web interface.

This lack of information makes it difficult to fully analyze the application performance and understand the real user experience. In particular, it is challenging to correlate user interactions with backend API processing.

The objective of this study is to analyze how the integration of frontend telemetry can improve the overall observability of the application. The work is based on an analysis of the existing system, a state-of-the-art review of application observability concepts and modern telemetry solutions, and an evaluation of different approaches for frontend instrumentation.

A hybrid OpenTelemetry solution, combining automatic instrumentation of network calls with manual instrumentation of functional steps, was implemented on a representative user journey. This implementation required solving several technical constraints specific to the browser environment, including the incompatibility with the gRPC protocol, the export format required by Aspire, CORS policy management through a Vite proxy, and the restriction of trace context propagation to internal requests only.

The results confirm an effective improvement in application behavior visibility. The correlation between user actions and backend processing works end to end, with frontend and backend spans sharing the same trace identifier in the Aspire dashboard. The instrumentation respects confidentiality constraints and does not introduce any noticeable performance degradation. This study demonstrates that it is possible, within an existing project and without modifying the backend architecture, to extend the observability chain to the browser and obtain a continuous view of application behavior, from the user click through to distributed processing in the microservices.

1 Introduction

1.1 Contexte général des applications web modernes

Les applications web modernes reposent sur des architectures distribuées combinant une interface utilisateur accessible depuis un navigateur, des API backend et différents composants de stockage ou d'échange de données. Cette organisation améliore la modularité, l'évolutivité et la maintenabilité des systèmes.

En contrepartie, elle augmente la complexité technique. Lorsqu'un dysfonctionnement ou une baisse de performance survient, il devient plus difficile d'en identifier rapidement l'origine, car une même action fonctionnelle peut mobiliser plusieurs composants.

L'essor des architectures de type Single Page Application renforce encore ce phénomène. Le navigateur ne se limite plus à afficher des pages : il gère une part importante des interactions, de la navigation et des échanges avec le backend. Dans ce contexte, une observation limitée au serveur ne suffit plus à comprendre complètement le comportement réel de l'application.

1.2 Importance de l'observabilité dans les systèmes informatiques

Pour analyser le comportement réel d'un système, les équipes techniques s'appuient sur l'observabilité [7]. Celle-ci repose sur la collecte et l'exploitation de logs, de métriques et de traces afin de détecter les anomalies, comprendre leur origine et accélérer le diagnostic.

Contrairement au simple monitoring, qui surveille surtout des indicateurs prédéfinis, l'observabilité vise à expliquer le comportement du système, y compris dans des situations non anticipées. Cette capacité est particulièrement importante dans les architectures distribuées, où un incident peut résulter de plusieurs traitements répartis entre différents services.

1.3 Présentation du sujet

Cette étude s'inscrit dans le cadre de mon alternance sur une application web du secteur de l'assurance. L'application repose sur un frontend web et plusieurs microservices backend orchestrés avec .NET Aspire.

Une solution d'observabilité [7] basée sur OpenTelemetry [1] est déjà en place côté backend. Elle permet de collecter des traces distribuées, des métriques et d'autres données techniques utiles à l'analyse des traitements serveur. En revanche, aucune télémétrie équivalente n'est actuellement déployée côté frontend.

Cette absence limite l'observabilité globale de l'application, car les équipes ne disposent pas d'une visibilité suffisante sur les actions utilisateur ni sur leur lien avec les traitements backend. Mon étude porte donc sur l'intégration d'une télémétrie frontend afin d'améliorer la continuité d'observation entre le navigateur et les microservices.

1.4 Organisation du rapport

Le rapport présente d'abord le contexte, le périmètre et les contraintes de l'étude, puis la problématique, les objectifs et les enjeux associés. Il propose ensuite une analyse de l'existant et un état de l'art sur l'observabilité applicative.

À partir de cette base, plusieurs solutions sont étudiées, comparées puis évaluées selon des critères de choix. Enfin, le rapport présente la mise en œuvre, la méthodologie de validation, les résultats obtenus et les perspectives d'amélioration.

2 Contexte et périmètre de l'étude

2.1 Présentation de l'entreprise et du projet

Ce travail est réalisé dans le cadre de mon alternance chez Sogeti, filiale du groupe Capgemini, au sein d'un projet mené dans le domaine de l'assurance.

Le projet s'inscrit dans une démarche de modernisation du système d'information assurance automobile. Il vise à faire évoluer les parcours de souscription et de gestion des contrats à travers une architecture plus modulaire, afin de faciliter les évolutions de l'application et les échanges entre composants.

2.2 Description de l'application étudiée

L'application étudiée fait partie d'une plateforme gérant différentes opérations du cycle de vie d'un contrat d'assurance automobile, comme la consultation d'informations véhicule, la réalisation de devis, la tarification ou certaines étapes de souscription et de gestion.

L'interface est utilisée dans plusieurs contextes, notamment via des parcours extranet ou intranet. Les actions réalisées dans le frontend déclenchent des appels vers plusieurs services backend chargés d'exécuter les traitements métier.

2.3 Architecture générale du système

L'application repose sur une architecture distribuée. Le frontend est développé avec React, TypeScript et Vite. Il constitue le point d'entrée principal des parcours utilisateurs.

Le backend est composé de plusieurs microservices .NET exposant des API métier. Ces services sont orchestrés avec .NET Aspire, qui centralise leur démarrage et leur configuration dans l'environnement de développement.

Cette organisation favorise l'évolution des composants, mais renforce aussi le besoin d'une observabilité transverse entre le navigateur et les services backend.

2.4 Périmètre de l'étude

Cette étude se concentre sur le frontend de l'application. L'objectif n'est pas d'instrumenter immédiatement l'ensemble du frontend, mais de concevoir et valider une approche sur un parcours utilisateur représentatif.

Le périmètre retenu couvre trois dimensions : la collecte d'événements pertinents côté navigateur, leur remontée dans une chaîne de télémétrie exploitable, et leur corrélation avec les traces déjà présentes côté backend.

2.5 Contraintes techniques et organisationnelles

L'intégration d'une solution de télémétrie frontend doit rester compatible avec l'environnement existant et ne pas remettre en cause l'architecture actuelle.

Trois contraintes principales structurent l'étude. La première est la compatibilité avec la solution d'observabilité backend déjà fondée sur OpenTelemetry. La deuxième est la maîtrise de l'impact sur les performances du navigateur. La troisième concerne la sécurité et la confidentialité, les données collectées ne devant pas contenir d'informations sensibles.

À cela s'ajoute une contrainte organisationnelle : la solution retenue doit rester simple à comprendre, à intégrer et à maintenir par les équipes.

3 Problématique et objectifs

3.1 Analyse de la situation actuelle

L'application dispose déjà d'une observabilité backend reposant sur OpenTelemetry. Cette instrumentation permet de collecter des données sur les requêtes HTTP, les appels interservices, l'exécution du runtime et certains accès aux bases de données. Les données sont exportées via OTLP et visualisées dans un outil adapté.

L'existant fournit donc une visibilité technique utile sur les traitements côté serveur. Cependant, cette observabilité commence à l'entrée du backend et ne couvre pas l'action utilisateur à l'origine de la requête.

3.2 Limites de l'observabilité actuelle

La principale limite de l'existant est son périmètre exclusivement backend. Les données disponibles permettent de comprendre ce que font les services, mais pas ce que fait l'utilisateur dans le navigateur avant qu'une requête n'atteigne le serveur.

Il n'est donc pas possible de relier précisément une interaction utilisateur à un traitement backend, ni d'évaluer correctement ce qui est réellement perçu dans l'interface. L'observabilité actuelle est utile, mais partielle.

3.3 Problématique de l'étude

La problématique de cette étude est la suivante :

- ➔ **Comment améliorer l'observabilité globale d'une application web en intégrant une télémétrie frontend, afin de permettre une corrélation efficace entre les interactions utilisateur et les traitements backend ?**

Cette problématique repose sur un écart clair entre l'existant et l'objectif visé : l'observabilité backend est déjà exploitable, mais elle ne permet pas de suivre un parcours complet depuis l'action utilisateur jusqu'aux microservices.

3.4 Objectifs du projet

Pour répondre à cette problématique, quatre objectifs sont retenus.

Le premier est d'instrumenter un parcours utilisateur représentatif dans le frontend afin de collecter au minimum un événement d'interface, un appel réseau vers le backend et un cas d'échec ou d'erreur observable.

Le deuxième est d'identifier une solution technique compatible avec l'architecture existante et avec l'instrumentation backend déjà fondée sur OpenTelemetry.

Le troisième est de vérifier, sur au moins un scénario complet, qu'une action utilisateur observée dans le navigateur peut être corrélée aux traitements backend correspondants.

Le quatrième est de vérifier que cette instrumentation respecte les contraintes du projet, en particulier l'absence de dégradation perceptible des performances et l'absence de remontée de données sensibles.

3.5 Enjeux techniques et métier

Les enjeux de cette étude sont à la fois techniques, opérationnels et métier. Sur le plan technique, il s'agit d'améliorer la compréhension globale du comportement de l'application et de faciliter le diagnostic des incidents. Sur le plan opérationnel, une meilleure corrélation frontend/backend peut améliorer la collaboration entre équipes. Enfin, sur le plan métier, une meilleure visibilité sur les parcours utilisateurs permet d'identifier plus facilement les points de friction perçus dans l'application.

4 Analyse de l'existant

4.1 Observabilité actuelle du backend

L'observabilité actuellement en place dans le projet est déployée côté backend, au sein de plusieurs microservices développés en .NET. L'analyse des fichiers Program.cs comme la [Figure 1/ 2](#) montre qu'OpenTelemetry est intégré dès le démarrage de l'application, avec une configuration homogène reposant notamment sur `AddOpenTelemetry()`, `WithMetrics()`, `WithTracing()` et `UseOtlpExporter()` [1]. Cette configuration a été observée dans plusieurs services backend, ce qui montre que la

démarche d'observabilité ne repose pas sur une instrumentation isolée, mais sur une logique commune à plusieurs composants de l'architecture.

Cette homogénéité constitue un premier point important. Elle signifie que l'observabilité backend n'est pas traitée comme un ajout ponctuel, mais comme une capacité transverse du système. Dans une architecture distribuée reposant sur plusieurs services, cette cohérence est essentielle pour suivre les traitements, comparer les comportements entre composants et reconstruire plus facilement le parcours d'une requête.

L'existant met donc en évidence une base d'observabilité déjà structurée et effectivement déployée côté serveur. Cette base fournit une visibilité réelle sur le comportement technique du backend, mais elle reste centrée sur le périmètre serveur, ce qui constitue à la fois sa force actuelle et sa principale limite au regard du sujet étudié.

4.1.1 Instrumentation OpenTelemetry

L'analyse du code de démarrage des microservices montre que l'instrumentation en place repose en grande partie sur les mécanismes standards fournis par OpenTelemetry. La configuration des ressources ajoute des informations d'identification du service, comme le nom du service, sa version, l'identifiant du processus et la version du framework .NET. Ces éléments facilitent l'exploitation des données en permettant de rattacher les traces et les métriques au bon composant.

Pour les métriques, la configuration observée active plusieurs instrumentations standards :

AddAspNetCoreInstrumentation() pour les traitements HTTP entrants,

AddHttpClientInstrumentation() pour les appels HTTP sortants, AddRuntimeInstrumentation() pour les métriques liées à l'exécution .NET, ainsi que AddProcessInstrumentation() pour les métriques de processus.

Pour les traces, la configuration active également AddAspNetCoreInstrumentation() pour les requêtes reçues, AddHttpClientInstrumentation() pour les appels vers d'autres services, ainsi que AddEntityFrameworkCoreInstrumentation() pour les accès aux bases de données. L'option SetDbStatementForText = true montre que le texte des requêtes SQL peut être remonté dans les spans observés, ce qui renforce la capacité d'analyse des traitements backend.

L'analyse révèle aussi la présence de mécanismes de filtrage et d'enrichissement. Certains endpoints techniques, comme /health, /favicon.ico, /metrics ou /swagger, sont exclus du traçage afin d'éviter de polluer les données avec des appels peu pertinents pour l'analyse fonctionnelle. On observe également un enrichissement de requête avec masquage partiel de la clé x-api-key, ce qui montre une prise en compte de la sécurité dans la collecte des données.

Cette instrumentation présente un avantage important : elle est standardisée, cohérente et relativement simple à déployer. En revanche, elle reste principalement orientée vers des données techniques d'infrastructure ou de traitement backend. Elle permet d'observer ce que font les services, mais elle ne capture pas encore le contexte fonctionnel côté utilisateur.

4.1.2 Collecte, export et visualisation des données

Les données de télémétrie produites par les microservices sont exportées via OTLP grâce à `UseOtlpExporter()`. Ce choix est cohérent avec une architecture moderne d'observabilité, dans la mesure où OTLP constitue le protocole standard d'export utilisé dans l'écosystème OpenTelemetry.

Dans l'environnement observé, les données sont ensuite visualisées dans le dashboard intégré à .NET Aspire. Les captures d'écran présentes en annexe montrent l'exploitation de deux types de vues : une vue orientée métriques ([figure 3](#)) et une vue orientée traces ([figure 4](#)). Dans la vue des métriques, on observe par exemple des indicateurs liés aux appels HTTP sortants, ce qui confirme que les métriques sont effectivement collectées et exploitables. Dans la vue des traces, plusieurs spans associés à différents services sont visibles, ce qui confirme le fonctionnement effectif du traçage distribué.

L'existant ne se limite donc pas à une configuration déclarative présente dans le code. Il produit bien des données observables en exécution, et ces données peuvent être consultées dans un outil de visualisation réellement utilisé dans le projet. Cet aspect renforce la fiabilité de l'analyse de l'existant.

En revanche, l'outillage observé semble principalement adapté à l'environnement de développement et d'analyse locale. À ce stade, l'existant étudié ne met pas en évidence une chaîne plus large de supervision de production. Cette limite n'est pas bloquante pour le sujet, mais elle montre que l'outillage actuel est surtout centré sur l'observation technique du backend dans un contexte de développement.

4.1.3 Propagation des traces

L'analyse du code montre qu'un mécanisme de propagation du contexte de trace est déjà mis en place. Les services utilisent `AddHeaderPropagation()` afin de propager certains en-têtes, notamment `traceparent`, ainsi qu'un en-tête de contexte utilisateur. Cette propagation permet de conserver le lien entre plusieurs étapes d'un même traitement lorsque la requête traverse plusieurs services.

Dans une architecture distribuée, ce mécanisme est essentiel. Il permet d'associer plusieurs spans à une même trace globale et donc de reconstruire le parcours d'un traitement entre les microservices. Les vues de traces observées dans Aspire confirment d'ailleurs que plusieurs services peuvent apparaître dans une même séquence de traitement.

L'existant montre donc que la corrélation backend-backend fonctionne déjà sur le plan technique. Il est possible de suivre un enchaînement d'appels entre services et d'identifier certaines latences ou certains points de blocage.

Cependant, cette propagation commence actuellement au niveau du backend. Le point d'entrée réel de l'action utilisateur, situé dans le navigateur, n'est pas inclus dans la chaîne de trace. La corrélation est donc incomplète : elle fonctionne entre services backend, mais elle ne remonte pas jusqu'à l'interaction ayant déclenché la requête.

4.2 Données actuellement collectées

À partir de l'analyse du code et des captures d'écran disponibles, plusieurs catégories de données actuellement collectées peuvent être identifiées.

La première catégorie correspond aux traces distribuées. Celles-ci permettent de suivre le parcours d'une requête à travers plusieurs microservices et constituent l'un des apports les plus visibles de la solution actuelle. Elles s'appuient sur l'instrumentation ASP.NET Core, HttpClient et Entity Framework Core.

La deuxième catégorie correspond aux échanges HTTP, qu'il s'agisse des requêtes entrantes traitées par les API ou des appels sortants réalisés par les services vers d'autres composants. Les métriques et les traces associées permettent d'observer des durées, des volumes d'activité et certaines caractéristiques techniques des appels.

La troisième catégorie concerne les accès aux bases de données. Grâce à l'instrumentation Entity Framework Core, les traitements impliquant des requêtes SQL peuvent être observés dans les traces, ce qui améliore sensiblement la capacité d'analyse des traitements métier exécutés côté backend.

Enfin, l'existant collecte également des métriques système et runtime liées à l'activité des processus et à l'exécution du framework .NET. Ces données permettent d'élargir la lecture du comportement des services au-delà du seul trafic applicatif.

L'ensemble fournit donc une visibilité technique réelle sur le backend. En revanche, il s'agit essentiellement de données d'exécution côté serveur. Elles permettent de comprendre ce que fait le système, mais pas de reconstituer directement ce que fait l'utilisateur dans l'interface.

4.3 Apports de la solution actuelle

La solution actuellement en place apporte déjà plusieurs bénéfices importants au projet.

Tout d'abord, elle fournit une observabilité homogène entre plusieurs microservices. Cette homogénéité facilite la lecture des comportements backend et permet une analyse plus cohérente à l'échelle de l'ensemble du système.

Ensuite, elle améliore les capacités de diagnostic. Les traces distribuées permettent d'identifier le service impliqué dans une latence, de suivre un enchaînement d'appels entre composants et de repérer certaines étapes critiques dans une chaîne de traitement. Les métriques offrent de leur côté une lecture plus synthétique de l'activité et du comportement de certains services.

La solution constitue également un socle technique déjà exploitable pour la suite de l'étude. Le protocole OTLP est en place, la propagation du contexte existe déjà, et l'outillage actuel permet de consulter les données générées. L'amélioration de l'observabilité ne part donc pas d'un système vierge, mais d'une base déjà structurée.

Enfin, cette base existante rend la problématique plus précise. Le sujet n'est pas de créer une observabilité complète à partir de zéro, mais d'étendre une observabilité backend déjà présente afin de la rendre plus continue et plus pertinente au regard du parcours utilisateur.

4.4 Limitations de la solution actuelle

Malgré ses apports, la solution actuelle présente plusieurs limites importantes.

La première limite est son périmètre exclusivement backend. Tout ce qui se passe dans le navigateur n'est pas tracé dans l'existant : navigation, clics, changements d'état de l'interface, temps perçu par

l'utilisateur ou encore contexte précis déclenchant un appel API. Les traces commencent donc trop tard par rapport au parcours réel.

La deuxième limite est l'absence de corrélation frontend-backend. Lorsqu'une erreur ou une latence est observée côté serveur, il n'est pas possible de connaître immédiatement l'action utilisateur qui en est à l'origine. Cette absence de lien rend la reproduction de certains incidents plus difficile, en particulier dans des parcours complexes ou multi-étapes.

La troisième limite tient à la nature essentiellement technique des données collectées. Les spans HTTP, les métriques de processus et les requêtes SQL sont utiles pour l'ingénierie backend, mais ils ne permettent pas à eux seuls d'évaluer correctement l'expérience utilisateur ni de relier directement un problème technique à une étape métier ou à un usage concret de l'interface.

Enfin, l'existant ne permet pas, à ce stade, d'observer certains indicateurs orientés usage, comme le temps réellement ressenti entre une action utilisateur et l'affichage d'un résultat, la fluidité d'un parcours ou l'apparition d'une erreur côté navigateur. Dans une application web moderne, ce type d'information devient pourtant essentiel pour disposer d'une observabilité réellement complète.

La limite principale de l'existant n'est donc pas l'absence d'instrumentation, mais le fait que cette instrumentation s'arrête au backend.

4.5 Synthèse

L'analyse de l'existant montre que le backend dispose déjà d'une observabilité technique réelle, cohérente et exploitable. Cette observabilité repose sur OpenTelemetry [1], sur une instrumentation homogène des microservices, sur l'export OTLP, sur la propagation du contexte de trace et sur la visualisation des données dans .NET Aspire.

L'existant constitue ainsi une base sérieuse pour le diagnostic technique côté serveur. Il permet de suivre les traitements backend, d'identifier certains ralentissements et de mieux comprendre les interactions entre plusieurs services.

En revanche, il reste incomplet au regard du fonctionnement global de l'application, car il ne couvre pas le point de départ réel du parcours : l'interface frontend et les interactions utilisateur. L'analyse met donc en évidence un écart net entre l'observabilité backend actuelle et le besoin d'une observabilité de bout en bout. C'est cet écart qui justifie la suite de mon étude.

5 Etat de l'art : observabilité des applications

5.1 Définition de l'observabilité

L'observabilité désigne la capacité à comprendre l'état interne d'un système à partir des données qu'il produit pendant son fonctionnement. Dans le domaine des applications web, elle permet d'analyser le comportement réel d'un système en production à partir d'informations telles que les logs, les métriques et les traces.

Comme l'illustre la [figure 5](#), l'observabilité ne se substitue pas au monitoring, mais le prolonge. Le monitoring permet principalement de surveiller des indicateurs connus, par exemple le temps de réponse, le taux d'erreur ou la disponibilité d'un service. L'observabilité va plus loin : elle cherche à expliquer pourquoi un problème se produit, en permettant de relier plusieurs données techniques et d'en reconstruire le contexte.

Cette distinction est particulièrement importante dans les architectures distribuées. Dans un système simple, la détection d'un seuil dépassé peut suffire à repérer un incident. Dans une architecture composée de plusieurs composants interconnectés, il devient en revanche nécessaire de disposer d'une approche plus complète, capable d'expliquer l'enchaînement des traitements et d'identifier les causes d'un comportement anormal.

L'observabilité repose donc sur une logique d'investigation. Elle ne consiste pas seulement à constater un problème, mais à donner les moyens de l'expliquer.

5.2 Logs, métriques et traces

L'observabilité applicative repose classiquement sur trois grandes catégories de données : les logs, les métriques et les traces comme l'illustre la [figure 6](#).

Les logs correspondent à des événements textuels générés par l'application. Ils permettent de conserver une trace détaillée d'une action, d'une erreur ou d'un événement métier. Ils sont particulièrement utiles pour comprendre le contexte précis d'un incident, mais leur exploitation peut devenir difficile lorsque le volume de données est important.

Les métriques sont des données numériques agrégées dans le temps. Elles permettent de suivre des indicateurs comme le temps de réponse, le nombre de requêtes, le taux d'erreur ou l'utilisation des ressources. Elles sont particulièrement adaptées à la surveillance du comportement général d'un système et à la détection rapide d'une anomalie.

Les traces distribuées permettent, quant à elles, de suivre le parcours complet d'une requête à travers différents composants. Dans une architecture microservices, elles jouent un rôle central, car elles rendent visible l'enchaînement des traitements entre plusieurs services, et permettent d'identifier les points de latence ou de rupture.

Ces trois catégories de données sont complémentaires. Les métriques permettent souvent de détecter un problème, les traces permettent d'en localiser l'origine dans une chaîne de traitement, et les logs permettent d'en approfondir la compréhension. Une observabilité efficace repose donc moins sur un seul type de donnée que sur leur articulation.

5.3 Observabilités dans les architectures distribuées

Les architectures distribuées, notamment fondées sur des microservices, introduisent une complexité importante dans le fonctionnement des applications. Contrairement à une architecture monolithique, une même fonctionnalité peut ici mobiliser plusieurs services distincts, chacun disposant de son propre cycle d'exécution, de ses dépendances et de ses échanges réseau.

Dans ce contexte, une requête fonctionnelle peut traverser plusieurs composants avant d'aboutir. Elle peut impliquer des appels API internes, des accès à des bases de données, voire des interactions avec d'autres systèmes. Cette distribution rend plus difficile l'identification de l'origine d'un

problème, en particulier lorsqu'une latence ou une erreur n'apparaît qu'à un endroit intermédiaire de la chaîne.

L'observabilité joue alors un rôle central. Les traces distribuées permettent de reconstituer le parcours complet d'une requête entre plusieurs services, tandis que les logs et les métriques viennent compléter cette lecture avec des informations détaillées sur le comportement local de chaque composant.

Cependant, l'observabilité des architectures distribuées soulève elle aussi plusieurs défis. Le premier est le volume de données produit, qui peut devenir important. Le deuxième est la nécessité de corréler correctement les informations entre plusieurs services. Le troisième est la capacité à relier des données très techniques à une lecture plus fonctionnelle du comportement du système.

L'efficacité de l'observabilité ne dépend donc pas uniquement de la quantité de données collectées, mais de la capacité à donner du sens à ces données dans une chaîne de traitement complète.

5.4 Présentation d'OpenTelemetry

OpenTelemetry est un projet open source soutenu par la Cloud Native Computing Foundation. Son objectif est de standardiser la collecte, le traitement et l'export des données de télémétrie dans les systèmes informatiques.

OpenTelemetry fournit des API, des bibliothèques et des mécanismes d'instrumentation permettant de collecter des logs, des métriques et des traces de manière cohérente, indépendamment du langage ou de la technologie utilisée. L'un de ses principaux apports réside dans la standardisation du modèle de données, qui facilite l'interopérabilité entre différents composants et différents outils.

Comme l'illustre la [figure 7](#), l'architecture OpenTelemetry [1] repose généralement sur trois éléments : les bibliothèques d'instrumentation intégrées dans les applications, le collecteur de télémétrie chargé de recevoir et de traiter les données, puis les outils de stockage, de visualisation et d'analyse. Cette organisation permet de dissocier la collecte des données de leur exploitation.

OpenTelemetry présente aussi l'avantage d'être compatible avec de nombreux environnements techniques, notamment .NET, Java, JavaScript ou Python. Il constitue ainsi un socle particulièrement adapté aux architectures distribuées et hétérogènes. Dans le cadre de mon étude, son intérêt est renforcé par le fait qu'il est déjà utilisé côté backend dans l'application analysée.

5.5 Panorama des solutions d'observabilité frontend

Avant d'aborder les principes de l'observabilité frontend, il est utile de situer les grandes catégories de solutions existantes dans l'industrie, afin de comprendre dans quel contexte s'inscrit le choix technique retenu dans ce projet.

Plusieurs plateformes commerciales proposent des solutions clés en main pour l'observabilité frontend. Datadog RUM, New Relic Browser ou encore Dynatrace [2] permettent de collecter automatiquement des données sur les sessions utilisateurs, les performances perçues et les erreurs côté navigateur. Ces solutions sont souvent faciles à intégrer et offrent des tableaux de bord riches, mais elles reposent sur des agents propriétaires et impliquent une dépendance forte à un éditeur tiers. Elles peuvent également soulever des questions en matière de confidentialité des données, selon les informations collectées et leur localisation.

D'autres outils, comme Sentry, sont davantage orientés vers la gestion des erreurs frontend. Ils permettent de capturer les exceptions JavaScript, d'en reconstituer le contexte et d'alerter les équipes en cas de problème. Leur périmètre reste cependant centré sur la gestion d'incidents, sans offrir une vision complète de la télémétrie distribuée.

Face à ces solutions, OpenTelemetry se distingue par son caractère ouvert, standardisé et indépendant de tout éditeur. Dans le cadre de ce projet, ce choix s'impose naturellement : le backend est déjà instrumenté avec OpenTelemetry, et l'objectif est d'étendre cette chaîne de télémétrie jusqu'au frontend sans introduire une nouvelle plateforme propriétaire. Cette cohérence technique facilite la corrélation des données entre les deux couches et limite la complexité de l'architecture globale.

5.6 Observabilité Frontend

L'observabilité frontend désigne l'ensemble des mécanismes permettant de collecter et d'analyser des données directement depuis le navigateur, afin de mieux comprendre le comportement d'une application côté client.

Elle complète naturellement l'observabilité backend. Alors que l'observabilité côté serveur permet de suivre les traitements techniques réalisés par les API et les services, l'observabilité frontend permet d'observer ce qui se passe au niveau de l'interface : navigation, interactions utilisateur, erreurs JavaScript, chargement des ressources, temps de rendu ou latence perçue.

Dans les applications web modernes, et en particulier dans les architectures de type Single Page Application, cette dimension est devenue essentielle. Le frontend ne se contente plus d'afficher des informations : il porte une part significative de la logique d'interface, orchestre des appels API, gère des états applicatifs et influence directement la perception de performance par l'utilisateur.

5.6.1 Deux approches complémentaires : Synthetic Monitoring et Real User Monitoring

Il existe deux grandes approches pour observer le comportement d'une application côté client. Le Synthetic Monitoring consiste à simuler des parcours utilisateurs à intervalles réguliers, depuis des environnements contrôlés, afin de détecter des régressions ou des indisponibilités. Cette approche est utile pour la surveillance proactive, mais elle ne reflète pas les conditions réelles d'utilisation.

Le Real User Monitoring, ou RUM, repose au contraire sur la collecte de données directement issues des sessions réelles des utilisateurs, depuis leur navigateur. Il permet de mesurer ce qui est effectivement vécu : temps de chargement réel, interactions concrètes, erreurs rencontrées en situation. Dans le cadre de ce projet, c'est cette approche qui est pertinente, car l'objectif est précisément de mieux comprendre le comportement réel de l'application en production, tel qu'il est perçu par les utilisateurs.

5.6.2 Les Web Vitals : un standard pour mesurer la performance perçue

Pour évaluer la performance perçue côté navigateur, Google a introduit les Core Web Vitals, qui sont devenus un standard largement adopté dans l'industrie. Ces indicateurs mesurent des dimensions directement liées à l'expérience utilisateur :

- Le LCP (Largest Contentful Paint) mesure le temps nécessaire pour afficher le principal élément visible de la page. Il reflète la perception de vitesse de chargement.

- Le FID (First Input Delay), désormais remplacé par l'INP (Interaction to Next Paint), mesure la réactivité de l'interface lors des interactions utilisateur.
- Le CLS (Cumulative Layout Shift) mesure la stabilité visuelle de la page, c'est-à-dire les déplacements inattendus d'éléments lors du chargement.
- Le TTFB (Time to First Byte) mesure le délai entre la requête et la réception du premier octet de réponse du serveur, ce qui permet de relier la performance perçue aux temps de traitement backend.

Ces indicateurs sont particulièrement utiles dans le cadre de l'observabilité frontend car ils permettent de quantifier objectivement ce que ressent l'utilisateur, au-delà des simples temps de réponse techniques. Leur intégration dans une chaîne de télémétrie permet de relier des indicateurs d'expérience utilisateur à des données techniques plus détaillées.

5.6.3 Les catégories de données collectées côté frontend

L'observabilité frontend permet de collecter plusieurs catégories d'informations pertinentes. Les événements d'interaction correspondent aux actions réalisées par l'utilisateur dans l'interface : clics, saisies, navigation entre écrans, validation d'étapes. Les performances perçues regroupent les indicateurs liés au chargement, au rendu et à la réactivité de l'interface, notamment les Web Vitals évoqués ci-dessus. Les erreurs côté navigateur incluent les exceptions JavaScript non gérées, les échecs de chargement de ressources ou les erreurs réseau. Enfin, les appels réseau initiés depuis le frontend constituent une catégorie essentielle, car ils représentent le lien direct entre l'interface utilisateur et les services backend.

Ces données complètent utilement la lecture backend, en apportant le point de vue utilisateur. L'un des principaux apports de l'observabilité frontend est justement la possibilité de relier une action utilisateur aux traitements backend qu'elle déclenche.

5.6.4 La corrélation session / trace : un enjeu central

Dans une architecture distribuée, relier une interaction frontend à une trace backend n'est pas trivial. Il est nécessaire de propager un contexte de trace depuis le navigateur jusqu'aux services, afin que les spans générés côté client et côté serveur puissent être associés à un même parcours cohérent.

Ce mécanisme repose sur la notion de propagation de contexte, standardisée par la spécification W3C Trace Context. Concrètement, lorsqu'un appel réseau est déclenché depuis le frontend, un en-tête traceparent est ajouté à la requête HTTP. Le backend peut alors récupérer ce contexte et rattacher ses propres spans à la même trace distribuée. La corrélation ne commence plus à l'entrée de l'API, mais dès l'action utilisateur dans le navigateur.

Cette continuité est au cœur de la problématique de ce projet. Sans elle, l'observabilité reste fragmentée : on sait ce que fait le serveur, mais on ignore ce qui a déclenché la requête, dans quel contexte, et ce que l'utilisateur a réellement perçu.

5.6.5 Les contraintes spécifiques au navigateur

La mise en place d'une observabilité frontend doit toutefois tenir compte de contraintes techniques propres à l'environnement navigateur, qui le distinguent d'un contexte serveur classique.

Premièrement, le protocole gRPC n'est pas supporté dans le navigateur. L'export des données de télémétrie doit donc passer par HTTP, généralement en JSON ou en protobuf sur HTTP/1.1 ou HTTP/2. Cette contrainte influence directement le choix de l'architecture d'export.

Deuxièmement, les politiques CORS (Cross-Origin Resource Sharing) et les Content Security Policy (CSP) peuvent bloquer les appels vers un collecteur externe si celui-ci n'est pas correctement configuré. Ces aspects doivent être anticipés dès la conception de l'architecture de collecte.

Troisièmement, l'instrumentation dans le navigateur introduit un coût en termes de performance. Le code d'observabilité s'exécute dans le même environnement que l'application, et tout ajout de traitement peut potentiellement impacter le temps de chargement ou la fluidité de l'interface. La collecte doit donc être conçue de manière légère et non bloquante.

Enfin, la collecte doit rester maîtrisée sur le plan de la confidentialité. Les données remontées ne doivent pas contenir d'informations personnelles ou sensibles. L'enjeu n'est donc pas de tout tracer, mais de tracer ce qui est réellement utile à l'analyse, en restant compatible avec les bonnes pratiques de protection des données.

6 Etudes de solutions possibles

6.1 Solutions de télémétrie frontend existantes

Dans le cadre de cette étude, j'ai choisi de limiter l'analyse aux solutions compatibles avec l'existant technique du projet. Cette orientation s'explique par deux éléments. D'une part, le backend de l'application est déjà instrumenté avec OpenTelemetry. D'autre part, l'objectif du projet n'est pas d'introduire une plateforme d'observabilité propriétaire supplémentaire, mais d'étendre la chaîne de télémétrie existante jusqu'au frontend, en conservant une logique standard, ouverte et cohérente avec l'architecture déjà en place.

Dans cet objectif, plusieurs approches compatibles avec OpenTelemetry peuvent être envisagées côté frontend. La première consiste à mettre en place une instrumentation automatique du navigateur, en s'appuyant sur les bibliothèques JavaScript d'OpenTelemetry. La documentation officielle montre en effet que l'instrumentation browser repose notamment sur `@opentelemetry/sdk-trace-web` et sur des modules d'instrumentation tels que `@opentelemetry/instrumentation-document-load`. Le registre officiel OpenTelemetry recense également des instrumentations dédiées au chargement de document, aux appels fetch, aux appels XMLHttpRequest, aux interactions utilisateur et aux exceptions web.

La deuxième approche consiste à privilégier une instrumentation manuelle, c'est-à-dire à créer explicitement des spans ou des événements sur des points fonctionnels choisis dans le frontend. La documentation OpenTelemetry [4] JavaScript rappelle que l'instrumentation correspond à l'ajout explicite de code d'observabilité dans l'application à l'aide du SDK et de l'API. Cette approche permet de cibler précisément les actions métier jugées pertinentes, mais elle demande davantage d'effort de développement et de normalisation.

La troisième approche, qui apparaît comme une solution intermédiaire, consiste à mettre en place une instrumentation hybride. Elle combine une instrumentation automatique des mécanismes techniques les plus standards du navigateur avec une instrumentation manuelle des événements métier ou des étapes fonctionnelles importantes du parcours utilisateur. Cette logique permet de conserver les avantages de l'automatisation tout en ajoutant un niveau de lecture plus proche du besoin fonctionnel du projet.

Dans la suite de cette partie, j'analyse donc ces trois approches compatibles avec OpenTelemetry, avant de les comparer au regard des contraintes de mon étude.

6.2 Instrumentation du navigateur

L'instrumentation automatique du navigateur présente un premier avantage important : elle permet de capter rapidement plusieurs catégories de signaux techniques sans devoir instrumenter manuellement chaque fonctionnalité. Les bibliothèques OpenTelemetry disponibles pour le navigateur permettent notamment de tracer le chargement initial du document, les appels réseau effectués depuis le frontend et certaines interactions utilisateur. Le registre officiel OpenTelemetry référence explicitement des instrumentations pour le chargement de document, fetch, XMLHttpRequest, les interactions utilisateur ainsi que les exceptions non gérées dans les applications web.

Cette approche reste cependant principalement orientée vers des événements techniques. Elle permet de savoir qu'un chargement de page a eu lieu, qu'un appel HTTP a été déclenché ou qu'une interaction a été captée, mais elle ne suffit pas toujours à exprimer clairement le contexte métier de l'action. Dans le cadre d'un parcours fonctionnel complexe, cette seule instrumentation peut donc être insuffisante pour relier proprement les traces à des étapes applicatives compréhensibles par les équipes.

L'instrumentation manuelle répond précisément à cette limite. En créant explicitement des spans ou des événements sur des points choisis de l'application, il devient possible d'associer la télémétrie à des actions fonctionnelles concrètes, par exemple l'ouverture d'un écran, la validation d'une étape de parcours ou le déclenchement d'une opération métier. Cette approche apporte une meilleure lisibilité fonctionnelle, mais elle suppose de bien définir à l'avance ce qu'il est utile de tracer et de maintenir cette instrumentation dans le temps.

L'approche hybride combine ces deux logiques. Les instrumentations automatiques permettent de couvrir rapidement le chargement du document et les appels réseau, tandis que les spans manuels apportent le contexte métier nécessaire à l'interprétation des traces. Dans le cadre de mon étude, cette approche est particulièrement pertinente, car l'objectif n'est pas uniquement de produire plus de données, mais de produire des données utiles pour comprendre le parcours utilisateur et le relier aux traitements backend. Elle permet donc de concilier couverture technique, lisibilité fonctionnelle et maîtrise du périmètre instrumenté.

La documentation officielle OpenTelemetry indique que l'instrumentation client dans le navigateur présente un niveau de maturité variable selon les composants. Le SDK de base (@opentelemetry/sdk-trace-web) ainsi que l'exporteur OTLP HTTP sont considérés comme stables. En revanche, certaines bibliothèques d'instrumentation automatique, comme celles dédiées aux interactions utilisateur ou aux exceptions, restent à un stade expérimental et sont susceptibles d'évoluer. Cette distinction est importante : elle ne remet pas en cause la viabilité de l'approche, mais elle invite à s'appuyer en priorité sur les composants stables pour les usages de production, et à traiter les instrumentations expérimentales avec davantage de prudence dans la maintenance à long terme.

6.3 Corrélation frontend / backend

La question de la corrélation entre le frontend et le backend est centrale dans mon sujet. L'objectif n'est pas seulement de produire des traces côté navigateur, mais de pouvoir reconstituer un enchaînement cohérent entre une action utilisateur, la requête HTTP envoyée au backend, puis les traitements distribués observés dans les microservices.

OpenTelemetry repose pour cela sur la propagation de contexte. La documentation officielle précise que cette propagation permet de corréler les signaux entre eux, même lorsqu'ils sont générés dans des processus ou des services distincts. Elle rappelle également que les propagateurs officiels utilisent par défaut les en-têtes définis par la spécification W3C Trace Context. Cette logique est particulièrement intéressante dans mon cas, car le backend observé propage déjà traceparent entre les microservices. L'ajout d'une instrumentation frontend compatible permet donc d'étendre cette chaîne de corrélation en amont, depuis le navigateur.

Concrètement, si une interaction utilisateur déclenche un span dans le frontend, puis qu'un appel fetch ou XMLHttpRequest est envoyé avec le contexte de trace associé, le backend peut récupérer ce contexte et rattacher la requête reçue à la même trace distribuée. La corrélation ne commence alors plus à l'entrée de l'API, mais dès l'action utilisateur. Cela répond directement à la limite mise en évidence dans l'analyse de l'existant, où la chaîne de trace était aujourd'hui complète entre microservices, mais incomplète entre le navigateur et le backend.

Sur le plan de l'export, la documentation OpenTelemetry JavaScript recommande l'usage d'un exporteur OTLP et précise que, dans le navigateur, l'export passe par HTTP/JSON ou HTTP/protobuf, le gRPC n'étant pas supporté. Elle rappelle également que cette architecture impose de tenir compte des politiques CSP, des en-têtes CORS et de l'exposition du point de terminaison de collecte. En pratique, cette contrainte conduit à privilégier une chaîne d'export OTLP maîtrisée, plutôt qu'un envoi direct non contrôlé.

Enfin, pour la visualisation locale et la démonstration, Aspire constitue un support cohérent avec cette approche. La documentation Aspire précise que le tableau de bord reçoit la télémétrie via OTLP et permet d'afficher traces, métriques et logs. Cette compatibilité avec OTLP est importante, car elle permet d'inscrire la démonstration frontend dans la continuité directe de l'observabilité déjà présente côté backend.

Le principe de cette corrélation est présenté en [figure 8](#).

6.4 Comparaison des solutions

Au regard du contexte de mon étude, les trois approches analysées présentent chacune des avantages et des limites.

L'instrumentation automatique du navigateur constitue la solution la plus rapide à mettre en place. Elle permet d'obtenir rapidement des données sur le chargement de page, les requêtes réseau et certaines interactions, avec un effort initial relativement limité. En revanche, elle reste surtout orientée vers une lecture technique du comportement du navigateur. Utilisée seule, elle ne permet pas toujours de relier clairement les traces à des étapes fonctionnelles précises du parcours utilisateur.

L'instrumentation manuelle offre au contraire une très bonne maîtrise du périmètre observé et du sens métier des données remontées. Elle est particulièrement adaptée lorsqu'il est nécessaire

d'associer explicitement la télémétrie à des étapes fonctionnelles ou à des événements applicatifs choisis. En revanche, son coût d'intégration est plus élevé, car elle demande un travail de conception, de développement et de maintenance plus important. Elle expose aussi davantage au risque d'hétérogénéité si les conventions d'instrumentation ne sont pas suffisamment cadrées.

L'approche hybride apparaît comme la plus équilibrée. Elle permet de tirer parti des instrumentations automatiques disponibles pour les signaux techniques déjà bien couverts, tout en ajoutant une instrumentation manuelle sur les étapes fonctionnelles réellement utiles à l'analyse. Dans le cadre de mon projet, cette solution présente plusieurs avantages décisifs : elle reste compatible avec OpenTelemetry, elle s'inscrit dans la continuité du backend déjà instrumenté, elle permet une corrélation frontend/backend plus lisible, et elle reste démontrable via OTLP dans Aspire. Elle constitue donc la solution la plus cohérente pour répondre à la problématique posée.

Cette analyse peut être synthétisée dans le tableau comparatif suivant.

Tableau 1 : Comparaison des solutions étudiées

Critère	Instrumentation automatique OpenTelemetry	Instrumentation manuelle OpenTelemetry	Approche hybride OpenTelemetry
Compatibilité avec l'existant backend	Très bonne	Très bonne	Très bonne
Facilité d'intégration initiale	Bonne	Moyenne à faible	Moyenne
Couverture technique navigateur	Bonne	Ciblée	Bonne
Lisibilité fonctionnelle	Moyenne	Très bonne	Très bonne
Corrélation frontend/backend	Bonne	Bonne à très bonne	Très bonne
Maîtrise du périmètre tracé	Moyenne	Très bonne	Très bonne
Maintenabilité	Bonne si périmètre simple	Dépend fortement des conventions	Bonne
Risque de sur-instrumentation	Moyen	Faible	Faible à moyen
Pertinence pour un parcours pilote	Bonne	Bonne	Très bonne
Pertinence pour la suite du projet	Moyenne	Moyenne	Très bonne

7 Choix de la solution

7.1 Critères de sélection

Le choix de la solution d'instrumentation frontend repose sur un ensemble de critères définis à partir des objectifs du projet, des contraintes identifiées et des enseignements tirés de l'analyse de l'existant. Ces critères ont été formulés de manière à être vérifiables et directement reliés au

contexte du projet. Ils doivent permettre de justifier le choix de la solution retenue de façon cohérente avec la problématique étudiée.

Le premier critère est la **compatibilité avec l'existant backend**. La solution retenue doit s'inscrire dans la continuité de l'instrumentation déjà en place côté serveur. Les microservices utilisent déjà OpenTelemetry avec export OTLP et propagation du contexte via l'en-tête traceparent. La solution frontend doit donc pouvoir s'intégrer dans cette même chaîne sans en modifier le fonctionnement.

Le deuxième critère est la **capacité de corrélation frontend/backend**. La solution doit permettre de relier explicitement une action utilisateur réalisée dans le navigateur aux traces distribuées côté backend. Ce critère est central, car il répond directement à la problématique de l'étude. Il implique que le contexte de trace puisse être propagé depuis le frontend vers les services backend, en s'appuyant sur le standard W3C Trace Context déjà utilisé dans l'architecture.

Le troisième critère est la **lisibilité fonctionnelle des données produites**. Les données de télémétrie collectées doivent être exploitables dans un contexte applicatif et métier. Il ne s'agit pas uniquement de produire des signaux techniques, mais de permettre aux équipes d'associer les traces à des étapes concrètes du parcours utilisateur.

Le quatrième critère est la **maîtrise du périmètre instrumenté**. La solution retenue doit permettre de contrôler précisément ce qui est tracé, afin d'éviter la remontée d'informations sensibles, de limiter le volume de données inutiles et de conserver une télémétrie réellement utile à l'analyse. Ce critère est directement lié aux contraintes de confidentialité et de performance identifiées précédemment.

Le cinquième critère est la **compatibilité avec l'environnement technique du projet**. La solution doit pouvoir être intégrée dans un frontend React/TypeScript outillé avec Vite, sans transformation lourde de l'architecture existante et sans introduire de dépendance propriétaire supplémentaire.

Le sixième critère est la maintenabilité et la simplicité d'intégration. La solution retenue doit rester compréhensible et facilement réutilisable par les équipes du projet. Elle doit s'appuyer sur des conventions claires, sur des composants suffisamment stables et sur une logique d'intégration réaliste dans la durée.

Enfin, le dernier critère est la **pertinence pour un parcours pilote représentatif**. Dans le cadre de cette étude, l'objectif n'est pas d'instrumenter l'ensemble du frontend dès la première étape, mais de valider une approche sur un parcours utilisateur concret. La solution retenue doit donc être adaptée à une mise en œuvre progressive, démontrable et suffisamment représentative pour être exploitée dans la suite du projet.

7.2 Analyse des solutions selon les critères

L'application de ces critères aux trois approches identifiées dans la partie précédente permet de confirmer et d'affiner les conclusions issues du tableau comparatif présenté en section 6.4.

L'instrumentation automatique répond correctement aux critères de compatibilité technique et de facilité d'intégration initiale. Elle permet d'obtenir rapidement des données sur les appels réseau, le chargement de page et certains événements standards du navigateur. Elle s'intègre naturellement dans une chaîne OpenTelemetry existante et constitue, de ce point de vue, une base intéressante. En revanche, elle présente des limites importantes sur les critères de lisibilité fonctionnelle et de maîtrise du périmètre. Utilisée seule, elle ne permet pas toujours d'associer les traces à des étapes

métier précises du parcours utilisateur, et le risque de produire des données nombreuses mais peu interprétables reste présent.

L'instrumentation manuelle répond très bien aux critères de lisibilité fonctionnelle et de maîtrise du périmètre. Elle permet de choisir précisément les points d'observation pertinents et d'associer chaque span à une action métier identifiée. Cette approche est particulièrement adaptée lorsqu'il est nécessaire de donner un sens applicatif clair aux données collectées. En revanche, son coût d'intégration est plus élevé. Elle suppose un effort plus important de conception, de développement et de maintenance, et sa qualité dépend fortement de la rigueur des conventions adoptées. Dans un contexte de projet, elle répond donc moins bien au critère de simplicité d'intégration si elle est utilisée seule.

L'approche hybride satisfait l'ensemble des critères de manière plus équilibrée. Elle s'appuie sur les composants stables du SDK OpenTelemetry JavaScript pour couvrir automatiquement les signaux techniques les plus courants, tout en réservant l'instrumentation manuelle aux étapes fonctionnelles réellement utiles à l'analyse. Cette combinaison permet d'assurer à la fois une couverture technique suffisante, une bonne lisibilité métier, une maîtrise du périmètre instrumenté et une compatibilité directe avec la chaîne OTLP déjà présente côté backend.

Elle répond également mieux au besoin de corrélation frontend/backend. Les appels réseau peuvent être instrumentés automatiquement, tandis que les spans manuels permettent de relier ces appels à des actions utilisateur identifiées dans le parcours. L'approche hybride apporte ainsi une lecture plus cohérente et plus exploitable du comportement global de l'application.

Enfin, cette approche apparaît comme la plus adaptée à une validation sur un parcours pilote. Elle permet de démontrer concrètement l'intérêt de la télémétrie frontend sans nécessiter une instrumentation exhaustive de l'ensemble de l'application. L'analyse selon les critères montre donc que l'approche hybride est la seule à répondre de manière satisfaisante à la fois aux contraintes techniques du projet, au besoin de corrélation frontend/backend et à l'exigence de lisibilité fonctionnelle des données.

7.3 Solution retenue

Au regard de l'analyse menée, la solution retenue est une **approche hybride OpenTelemetry**, combinant instrumentation automatique et instrumentation manuelle dans le frontend React/TypeScript de l'application.

Ce choix s'impose pour plusieurs raisons convergentes. Tout d'abord, cette solution est la seule à satisfaire simultanément l'ensemble des critères définis, notamment la compatibilité avec l'existant, la corrélation frontend/backend, la lisibilité fonctionnelle et la maîtrise du périmètre instrumenté. Ensuite, elle s'inscrit directement dans la continuité de l'observabilité backend déjà en place : les microservices exportent déjà leurs données via OTLP et propagent le contexte de trace via traceparent. Une instrumentation frontend compatible avec ce même standard permet donc d'étendre cette chaîne en amont, depuis le navigateur, sans modifier l'architecture existante.

Cette solution est également cohérente avec le périmètre de l'étude. L'objectif n'est pas d'industrialiser immédiatement une couverture complète de tout le frontend, mais de valider une approche pertinente sur un parcours utilisateur pilote représentatif. Dans cette perspective,

l'approche hybride offre un niveau de couverture suffisant tout en restant réaliste en termes d'intégration et de maintenance.

Sur le plan technique, cette solution repose sur plusieurs composants OpenTelemetry adaptés au navigateur. Le SDK de base est fourni par `@opentelemetry/sdk-trace-web`. L'export des traces vers le point de collecte OTLP d'Aspire repose sur `@opentelemetry/exporter-trace-otlp-proto`. L'instrumentation automatique des appels réseau est assurée par `@opentelemetry/instrumentation-fetch` et `@opentelemetry/instrumentation-xml-http-request`, tandis que le chargement initial du document est pris en charge par `@opentelemetry/instrumentation-document-load`.

L'instrumentation manuelle est, quant à elle, réalisée directement à l'aide de l'API OpenTelemetry JavaScript sur les étapes fonctionnelles retenues dans le parcours pilote.

La solution retenue n'a donc pas été choisie uniquement parce qu'elle est techniquement réalisable, mais parce qu'elle constitue le meilleur compromis entre cohérence avec l'existant, pertinence fonctionnelle, capacité de corrélation, faisabilité de mise en œuvre et perspective d'évolution.

7.4 Architecture de la solution proposée

L'architecture de la solution proposée s'appuie sur la chaîne de télémétrie déjà en place côté backend, et l'étend en y intégrant le frontend comme nouveau point d'entrée de la trace distribuée. Cette architecture est présentée en [figure 9](#).

Côté frontend, le SDK OpenTelemetry JavaScript est initialisé au démarrage de l'application React. Il configure un TracerProvider associé à une ressource identifiant le service frontend, notamment par son nom et sa version. Les instrumentations automatiques sont enregistrées dès l'initialisation et prennent en charge les appels fetch et XMLHttpRequest émis par l'application, en y injectant automatiquement l'en-tête traceparent conformément à la spécification W3C Trace Context. Les spans manuels sont créés sur les étapes fonctionnelles choisies du parcours instrumenté, en utilisant directement l'API du tracer. L'ensemble des spans produits est exporté via un exporteur OTLP HTTP vers le point de collecte exposé par .NET Aspire.

Côté backend, la configuration existante reste inchangée. Les microservices ProductionVehicule, DevisContratsVehicule et CatalogueVehicules reçoivent les requêtes HTTP émises par le frontend, qui portent désormais l'en-tête traceparent généré par le SDK frontend. Grâce au mécanisme `AddHeaderPropagation()` déjà configuré dans les services, ce contexte de trace est automatiquement propagé entre les microservices. Les spans backend sont ainsi rattachés à la même trace distribuée que les spans frontend, ce qui permet de reconstituer le parcours complet depuis l'action utilisateur jusqu'aux traitements exécutés dans les services.

Côté visualisation, le dashboard .NET Aspire [3] reçoit l'ensemble des données de télémétrie via OTLP, qu'elles proviennent du frontend ou des microservices backend. Cette convergence vers un point de visualisation unique permet d'afficher des traces distribuées couvrant l'intégralité du parcours observé, du navigateur jusqu'aux traitements backend et aux accès aux bases de données. Cette architecture présente donc l'avantage de conserver un point unique de démonstration, de lecture et d'analyse, sans ajout d'outillage supplémentaire.

Du point de vue des contraintes, l'export OTLP depuis le navigateur passe par HTTP, ce qui est cohérent avec les limitations propres à l'environnement navigateur identifiées en section 5.6.5. La question du CORS est prise en compte dans l'environnement de développement utilisé pour l'étude,

ce qui permet de valider l'approche sans remise en cause de l'architecture réseau existante. Enfin, aucune donnée sensible n'est remontée dans les spans : les instrumentations automatiques sont configurées de manière maîtrisée, et les spans manuels sont définis pour ne porter que des informations fonctionnelles non nominatives.

Cette architecture ne remplace donc pas l'existant : elle le prolonge en ajoutant le navigateur comme point de départ de la chaîne d'observabilité. Elle fournit ainsi une base cohérente pour la mise en œuvre et la validation présentées dans la partie suivante.

8 Mise en œuvre et méthodologie de validation

8.1 Environnement technique

La mise en œuvre de l'instrumentation frontend s'inscrit dans l'environnement de développement déjà utilisé dans le projet. L'ensemble des services est orchestré par .NET Aspire, qui centralise le démarrage et la configuration de tous les composants applicatifs en une seule commande, sans distinction entre les services frontend et backend. Le lancement du projet s'effectue via la commande suivante :

```
dotnet run --project AspireProductionVehicule/AspireProductionVehicule.csproj --launch-profile "Dev-Matmut"
```

Cette commande démarre simultanément les trois microservices backend (BS-Production_Vehicule, CS-Devis_Contrats_Vehicule et CS-Catalogue_Vehicules), ainsi que les deux frontends du projet : FIA-Production_Vehicule, développé en React et TypeScript avec Vite sur le port 3000, et FIE-Production_Vehicule, développé en Next.js sur le port 3001. Le dashboard Aspire est accessible à l'adresse <https://localhost:17006> et constitue le point central de visualisation des données de télémétrie collectées par l'ensemble des services.

Le frontend concerné par cette étude est FIA-Production_Vehicule. Il s'agit d'une application React/TypeScript outillée avec Vite, qui constitue l'interface principale des parcours utilisateurs étudiés. L'environnement d'exécution JavaScript repose sur Node.js v22.19.0 et npm 10.9.3.

Le tableau suivant récapitule les principaux éléments de l'environnement technique utilisé dans cette mise en œuvre.

Tableau 2 : Environnement technique de mise en œuvre

Composant	Valeur
Node.JS	v22.19.0
npm	v 10.9.3
Framework	React / TypeScript / Vite
Orchestrateur	.NET Aspire
Profil de lancement	Dev-Matmut
Dashboard aspire	https://localhost:17006
Endpoint OTLP gRPC (backend)	Port 21011
Endpoint OTLP HTTP (frontend)	Port 21012

8.2 Résolution du problème d'export depuis le navigateur

8.2.1 Contrainte fondamentale : le navigateur ne supporte pas gRPC

Les trois microservices backend envoient déjà leurs traces vers le dashboard Aspire via le protocole gRPC sur le port 21011. L'objectif initial était de faire apparaître le frontend React dans le même onglet Traces, afin d'obtenir une vue de bout en bout couvrant l'intégralité de la chaîne : frontend -> BS -> CS.

Cependant, un navigateur ne peut pas émettre de requêtes gRPC. Le protocole gRPC repose sur HTTP/2 avec des fonctionnalités de streaming binaire qui ne sont pas accessibles depuis les API réseau disponibles dans l'environnement navigateur. L'export des traces frontend ne peut donc pas emprunter le même chemin que les services backend. Il a été nécessaire de trouver un chemin alternatif pour faire parvenir les traces du navigateur jusqu'à Aspire.

8.2.2 Ouverture d'un second endpoint OTLP dans Aspire

Pour résoudre cette contrainte, un second endpoint OTLP a été configuré dans Aspire, cette fois en HTTP/1.1 sur le port 21012. Cette configuration s'effectue par l'ajout d'une variable d'environnement dans le profil de lancement du projet. Ce second endpoint est compatible avec les requêtes HTTP classiques que le navigateur est capable d'émettre, tout en restant un récepteur OTLP standard capable de traiter les traces, les métriques et les logs.

Il est important de noter que cet endpoint HTTP n'accepte que le format protobuf. Les tentatives d'envoi au format JSON ont échoué, Aspire refusant systématiquement les données dans ce format. Cette contrainte a directement orienté le choix de l'exporteur utilisé côté frontend.

8.2.3 Mise en place d'un proxy Vite pour relayer les traces

L'export direct depuis le navigateur vers le port 21012 d'Aspire se heurte à une seconde difficulté : les politiques CORS. Le navigateur exécute l'application sur le port 3000, tandis que l'endpoint OTLP est exposé sur le port 21012. Cette différence d'origine déclenche des requêtes de pré-vérification (preflight) que l'endpoint Aspire n'est pas configuré pour gérer.

Pour contourner cette contrainte, un middleware proxy a été mis en place dans le serveur de développement Vite. Le navigateur envoie ses traces à l'adresse `/otlp/v1/traces` sur sa propre origine, ce qui évite toute problématique CORS puisque la requête reste sur le même domaine. Côté serveur, le middleware Vite intercepte ces requêtes et les relaie vers le port 21012 d'Aspire.

Ce middleware assure trois opérations essentielles lors du relais. Premièrement, il supprime les cookies du dashboard Aspire que le navigateur envoie automatiquement avec la requête, car l'endpoint OTLP refuse les requêtes contenant ces cookies. Deuxièmement, il ajoute la clé API qu'Aspire injecte dans les variables d'environnement du processus Vite au moment du lancement, cette clé étant nécessaire pour que l'endpoint accepte les données. Troisièmement, il transmet fidèlement le Content-Type protobuf sans le modifier, afin que l'endpoint OTLP puisse décoder correctement les données reçues.

Ce même proxy intercepte également les requêtes vers `/otlp/v1/metrics`, ce qui permet d'exporter les métriques frontend selon le même mécanisme, sans configuration supplémentaire.

8.3 Instrumentation du frontend

L'instrumentation du frontend repose sur l'approche hybride retenue en section 7, combinant une instrumentation automatique des appels réseau avec une instrumentation manuelle sur les étapes fonctionnelles du parcours pilote sélectionné.

8.3.1 Parcours pilote retenu

Le parcours instrumenté est la recherche d'un lieu de garage par code postal, accessible depuis l'écran "Votre véhicule et vous" à la route /devis, étape 5 du formulaire multi-étape, dans la section "Usage du Véhicule". Ce parcours a été retenu car il est représentatif d'une interaction utilisateur complète : il implique une action explicitement déclenchée par l'utilisateur, un appel API vers un service backend, l'affichage d'un résultat et une validation finale. Il met donc en jeu l'ensemble de la chaîne que l'on cherche à observer, depuis le clic dans le navigateur jusqu'au traitement distribué côté serveur.

Le déroulement du parcours est le suivant. L'utilisateur clique sur l'icône de recherche de lieu de garage dans le composant FormUsageVehicule.tsx, ce qui ouvre une popup (PopupLieuGarageContent.tsx). Il saisit un code postal et/ou un nom de ville, puis déclenche la recherche en cliquant sur le bouton de recherche. Cette action provoque un appel GET /api/v1/communes vers le service bs-gestion-contrats-iard, avec les paramètres codePostal et nom en query string. Les résultats s'affichent dans un tableau, l'utilisateur sélectionne une commune et valide via le bouton "Reporter le lieu de garage", ce qui met à jour les champs du formulaire et ferme la popup. Les différentes étapes de ce parcours sont illustrées en figure 10 et 11 en [annexe](#).

8.3.2 Installation des packages

L'instrumentation repose sur les packages OpenTelemetry suivants, ajoutés aux dépendances du projet frontend :

```
npm install @opentelemetry/sdk-trace-web @opentelemetry/exporter-trace-otlp-proto
@opentelemetry/instrumentation-fetch @opentelemetry/instrumentation-xml-http-request
@opentelemetry/instrumentation-document-load @opentelemetry/context-zone
@opentelemetry/sdk-metrics @opentelemetry/exporter-metrics-otlp-proto
```

Le package @opentelemetry/sdk-trace-web constitue le SDK de base pour le navigateur. L'exporteur @opentelemetry/exporter-trace-otlp-proto prend en charge l'envoi des spans en protobuf vers le proxy Vite, qui les relaie ensuite à l'endpoint OTLP HTTP d'Aspire. Le choix du format protobuf, plutôt que JSON, s'impose car l'endpoint HTTP d'Aspire refuse le format JSON.

Les packages @opentelemetry/instrumentation-fetch et @opentelemetry/instrumentation-xml-http-request assurent l'instrumentation automatique des appels réseau, tandis que @opentelemetry/instrumentation-document-load couvre le chargement initial du document. Le package @opentelemetry/context-zone assure la propagation correcte du contexte de trace dans l'environnement asynchrone du navigateur.

Les packages `@opentelemetry/sdk-metrics` et `@opentelemetry/exporter-metrics-otlp-proto` permettent la collecte et l'export de métriques frontend, selon le même chemin d'export que les traces.

8.3.3 Initialisation du SDK

Le SDK OpenTelemetry est initialisé dans un fichier dédié `otel.ts`, chargé au démarrage de l'application React avant le rendu de l'interface. Ce fichier est importé en tout premier dans le point d'entrée de l'application (`main.tsx`) afin de capturer les requêtes réseau dès le démarrage. La configuration complète de ce fichier est présentée en figure 12 en [annexe](#), et l'import dans `main.tsx` est visible en figure 13.

Cette initialisation configure deux éléments principaux.

Le premier est un `TracerProvider` associé à une ressource identifiant le service frontend par son nom (`fia-production-vehicule`) et sa version. Ce provider est associé à un exporteur OTLP protobuf pointant vers `/otlp/v1/traces`, c'est-à-dire la même origine que l'application, le proxy Vite se chargeant du relais vers Aspire. Les instrumentations automatiques sont enregistrées dès l'initialisation et prennent en charge les appels `fetch`, les requêtes `XMLHttpRequest` et le chargement du document.

Le second est un `MeterProvider` configuré pour exporter les métriques toutes les 10 secondes en protobuf vers `/otlp/v1/metrics`, via le même proxy Vite. Ce provider expose quatre métriques prédéfinies, prêtes à être utilisées dans les composants de l'application, comme le montre la figure 14 en [annexe](#). Le tableau suivant récapitule ces métriques.

Tableau 3: Métriques frontend définies dans `otel.ts`

Métriques	Type	Description
<code>Fia.page_views</code>	Compteur	Nombre de pages vues
<code>Fia.user_actions</code>	Compteur	Nombre d'actions utilisateur
<code>Fia.errors</code>	Compteur	Nombre d'erreurs frontend
<code>Fia.api_call_duration</code>	Histogramme	Durée des appels API en ms

8.3.4 Restriction de la propagation du traceparent

L'instrumentation automatique des appels `fetch` injecte par défaut un en-tête `traceparent` dans toutes les requêtes HTTP sortantes, conformément à la spécification W3C Trace Context. Ce mécanisme est essentiel pour la corrélation frontend/backend, car il permet au service backend de rattacher ses propres spans à la même trace distribuée que le frontend.

Cependant, cette injection systématique pose un problème avec les API externes à l'application. Lorsqu'un en-tête `traceparent` est ajouté à une requête vers un domaine tiers, le navigateur déclenche une requête `preflight` CORS pour vérifier si le serveur accepte cet en-tête personnalisé. Les API externes qui ne connaissent pas cet en-tête rejettent le `preflight`, ce qui provoque l'échec de la requête.

Pour résoudre ce problème, la propagation du `traceparent` a été restreinte aux seules requêtes destinées à `localhost`, c'est-à-dire celles qui passent par le proxy Aspire vers les microservices

backend. Les requêtes vers des domaines externes ne portent ainsi aucun en-tête de trace, ce qui préserve leur fonctionnement normal.

8.3.5 Instrumentation automatique

Les instrumentations automatiques fetch et XMLHttpRequest prennent en charge l'ensemble des appels réseau émis par l'application, y compris ceux réalisés via React Query lors de l'appel GET /api/v1/communes. Pour les requêtes destinées à localhost, ces instrumentations injectent automatiquement l'en-tête traceparent, conformément à la restriction définie en section 8.3.4. C'est ce mécanisme qui permet au service backend de récupérer le contexte de trace initié dans le navigateur et de rattacher ses propres spans à la même trace distribuée.

L'instrumentation document-load capture quant à elle les événements liés au chargement initial de la page, fournissant des données sur les temps de chargement du document et des ressources associées.

8.3.6 Instrumentation manuelle

Des spans manuels sont créés sur les étapes fonctionnelles identifiées dans le parcours pilote. Quatre points d'instrumentation ont été retenus, chacun correspondant à une action utilisateur précise dans le parcours de recherche de lieu de garage. Le tableau suivant détaille ces spans.

Tableau 4 : Spans manuels définis dans le parcours pilote

Span	Fichier source	Action utilisateur
garage.popup.open	FormUsageVehicule.tsx	L'utilisateur ouvre la popup de lieu de garage
garage.search.triggered	PopupLieuGarageContent.tsx	L'utilisateur clique sur "Rechercher"
garage.search.results	PopupLieuGarageContent.tsx	Les résultats de la recherche arrivent
garage.location.confirmed	PopupLieuGarageContent.tsx	L'utilisateur valide son choix de commune

Le pattern d'instrumentation est identique pour chaque point. Le tracer, exporté depuis otel.ts, est importé dans le composant React concerné. Un appel à tracer.startActiveSpan() crée un span au moment de l'action, et le span est terminé immédiatement après l'exécution de la logique associée. Le code correspondant dans FormUsageVehicule.tsx est présenté en figure 15 en [annexe](#), et celui dans PopupLieuGarageContent.tsx en [figure](#) 16.

Les spans portent des attributs fonctionnels non nominatifs, tels que le nombre de communes retournées par l'API ou le fait qu'une sélection ait été effectuée, sans jamais remonter de données personnelles telles que le code postal saisi ou le nom de l'utilisateur.

Les requêtes HTTP GET /api/v1/communes sont quant à elles tracées automatiquement par la FetchInstrumentation, sans nécessiter de code d'instrumentation manuel. Ce découplage entre instrumentation manuelle des actions métier et instrumentation automatique des appels réseau

illustre concrètement le fonctionnement de l'approche hybride retenue. Pour tracer d'autres actions utilisateur dans l'application à l'avenir, il suffit d'ajouter le même pattern à l'endroit voulu dans le code.

8.4 Collecte des données de télémétrie

8.4.1 Chaîne d'export des traces

Les données produites par l'instrumentation frontend suivent un chemin d'export en trois étapes. Le SDK OpenTelemetry dans le navigateur sérialise les spans en protobuf et les envoie à l'adresse `/otlp/v1/traces` sur la même origine que l'application. Le middleware proxy dans le serveur Vite intercepte cette requête, supprime les cookies, ajoute la clé API Aspire et relaie le tout vers le port 21012 d'Aspire. L'endpoint OTLP HTTP d'Aspire reçoit les données et les intègre dans le dashboard aux côtés des traces des microservices backend.

Pour les métriques, le chemin est identique : le MeterProvider exporte en protobuf vers `/otlp/v1/metrics`, le proxy Vite relaie vers Aspire, et les métriques apparaissent dans l'onglet Metrics du dashboard.

8.4.2 Corrélation avec le backend

Côté backend, la configuration existante reste intégralement inchangée. Les microservices reçoivent les requêtes HTTP émises par le frontend, qui portent désormais l'en-tête `traceparent` généré par le SDK OpenTelemetry JavaScript. Le mécanisme `AddHeaderPropagation()` déjà configuré dans chaque service propage automatiquement ce contexte entre les microservices. Les spans backend sont ainsi rattachés à la même trace distribuée que les spans frontend, sans aucune modification du code serveur.

L'ensemble des données de télémétrie, qu'elles proviennent du frontend ou des microservices backend, convergent vers le dashboard Aspire. Ce point de collecte unique permet d'afficher dans une même vue les spans frontend et les spans backend associés à un même parcours utilisateur, offrant ainsi une lecture continue de la chaîne d'exécution depuis le navigateur jusqu'aux traitements distribués côté serveur.

8.5 Méthodologie de validation

La méthodologie de validation repose sur une vérification manuelle dans le dashboard Aspire, organisée autour des quatre objectifs définis en section 3.4. Chaque objectif fait l'objet d'un critère de validation précis et observable, dont le résultat attendu est clairement défini avant l'exécution des tests.

8.5.1 Validation de l'objectif 1 : Collecte d'événements frontend

Le premier critère consiste à vérifier que les spans produits par l'instrumentation frontend apparaissent bien dans le dashboard Aspire après exécution du parcours pilote. Le parcours est joué dans le navigateur, et la vue Traces du dashboard est consultée immédiatement après. La présence d'au moins un span identifié avec le nom de service `fia-production-vehicule`, couvrant l'ouverture de

la popup, le déclenchement de la recherche, l'appel réseau GET /api/v1/communes et la validation du lieu de garage, constitue le résultat attendu validant cet objectif.

8.5.2 Validation de l'objectif 2 : Corrélation frontend/backend

Le deuxième critère est le plus important au regard de la problématique de l'étude. Il consiste à vérifier que le span frontend correspondant à l'appel GET /api/v1/communes et le span backend correspondant au traitement de cette requête dans le service bs-gestion-contrats-iard partagent le même traceId. Dans la vue des traces du dashboard Aspire, une trace distribuée correctement corrélée doit afficher une séquence de spans couvrant à la fois le navigateur et le service backend, tous rattachés au même identifiant de trace. Ce résultat démontre que la chaîne de corrélation fonctionne de bout en bout, depuis l'action utilisateur jusqu'au traitement distribué côté serveur.

8.5.3 Validation de l'objectif 3 : Absence de données sensibles

Le troisième critère consiste à inspecter le détail des spans produits dans Aspire et à vérifier qu'aucun attribut ne contient d'information personnelle ou sensible. Les valeurs de code postal et de nom de ville saisies par l'utilisateur ne doivent pas apparaître dans les attributs des spans. Seuls des attributs fonctionnels agrégés et non nominatifs sont attendus.

8.5.4 Validation de l'objectif 4 : Absence de dégradation des performances

Le quatrième critère consiste à vérifier que l'ajout de l'instrumentation n'introduit pas de dégradation perceptible sur le parcours instrumenté. Le temps de réponse de l'appel GET /api/v1/communes est mesuré dans les outils de développement du navigateur, onglet Réseau, en comparant une exécution sans instrumentation et une exécution avec instrumentation active. L'absence de différence significative constitue le résultat attendu validant cet objectif.

8.6 Mise en place des tests

La validation de l'instrumentation repose sur une série de quatre scénarios de tests manuels, exécutés dans l'environnement de développement local avec le profil de lancement Dev-Matmut.

8.6.1 Scénario 1 : Vérification de la collecte des spans frontend

Ce scénario valide que le SDK OpenTelemetry est correctement initialisé et que les spans sont bien exportés vers Aspire. Les étapes sont les suivantes : lancer le projet via Aspire avec le profil Dev-Matmut, ouvrir le frontend dans le navigateur, naviguer jusqu'à l'écran /devis à l'étape 5, ouvrir la popup de recherche de lieu de garage, puis consulter la vue Traces dans le dashboard Aspire. Le résultat attendu est la présence d'au moins un span identifié avec le nom de service fia-production-vehicule dans la liste des traces.

8.6.2 Scénario 2 : Vérification de la corrélation frontend/backend

Ce scénario valide le mécanisme de propagation du contexte de trace, qui constitue le cœur de la problématique de l'étude. Les étapes sont : saisir un code postal dans la popup de recherche, cliquer sur le bouton de recherche pour déclencher l'appel GET /api/v1/communes, attendre l'affichage des résultats, puis ouvrir dans Aspire la trace correspondant à cet appel. Le résultat attendu est une trace distribuée contenant à la fois les spans frontend du parcours et les spans backend du service bs-gestion-contrats-iard, tous partageant le même traceId.

8.6.3 Scénario 3 : Vérification de l'absence de données sensibles

Ce scénario consiste à effectuer une recherche complète avec un code postal de test, puis à inspecter dans Aspire le détail de chaque attribut des spans produits. Le résultat attendu est l'absence totale de valeurs saisies par l'utilisateur dans les attributs, confirmant que l'instrumentation respecte les contraintes de confidentialité.

8.6.4 Scénario 4 : Vérification de l'impact sur les performances

Ce scénario consiste à mesurer le temps de réponse de l'appel GET /api/v1/communes dans l'onglet Réseau des outils de développement du navigateur, d'abord sans instrumentation active, puis avec instrumentation active. L'absence de dégradation perceptible constitue le résultat attendu.

Ces quatre scénarios couvrent l'ensemble des objectifs définis en section 3.4 et permettent de valider ou d'invalidier la solution retenue de manière structurée et reproductible. Leurs résultats sont analysés dans la partie suivante.

9 Résultats et analyse

9.1 Données collectées

L'exécution du parcours pilote dans l'environnement de développement a permis de confirmer que l'instrumentation produit effectivement des données exploitables dans le dashboard Aspire.

Dans l'onglet Traces, le service fia-production-vehicule apparaît comme une ressource à part entière, aux côtés des trois microservices backend déjà présents. Les spans manuels définis dans le parcours pilote sont visibles et correctement identifiés : garage.popup.open, garage.search.triggered, garage.search.results et garage.location.confirmed. Chacun de ces spans correspond à une action utilisateur précise dans le parcours de recherche de lieu de garage, ce qui confirme que l'instrumentation manuelle fonctionne comme prévu. Ce résultat est observable dans la [figure 17](#) en annexe, qui présente la liste des traces du service fia-production-vehicule dans le dashboard Aspire.

Les appels réseau émis par l'application sont également tracés automatiquement par la FetchInstrumentation. L'appel GET /api/v1/communes apparaît dans les traces sans qu'aucun code d'instrumentation manuel n'ait été nécessaire pour le capturer, ce qui valide le fonctionnement de l'instrumentation automatique.

Dans l'onglet Metrics, les compteurs fia.page_views, fia.user_actions, fia.errors et l'histogramme fia.api_call_duration apparaissent après environ 10 secondes de navigation, comme le montre la [figure 18](#) en [annexe](#). Le service fia-production-vehicule est bien identifié comme source des

métriques, confirmant que la chaîne d'export des métriques fonctionne correctement via le proxy Vite.

Le scénario 1 de validation est donc satisfait : l'instrumentation frontend produit bien des traces et des métriques exploitables dans le dashboard Aspire.

9.2 Corrélation frontend / backend

Le résultat le plus significatif de cette mise en œuvre concerne la corrélation entre les spans frontend et les spans backend. Lorsque l'utilisateur déclenche une recherche de commune dans la popup, l'appel GET /api/v1/communes est émis par le navigateur avec un en-tête traceparent généré par le SDK OpenTelemetry JavaScript. Le service bs-gestion-contrats-iard reçoit cette requête, récupère le contexte de trace et rattache ses propres spans à la même trace distribuée.

Dans le dashboard Aspire, cette corrélation est directement observable. En ouvrant la trace correspondant à l'appel de recherche de communes, on obtient une vue distribuée complète montrant l'enchaînement des spans depuis le frontend jusqu'aux microservices backend. Les spans du service fia-production-vehicule et ceux du service bs-gestion-contrats-iard partagent le même traceId, ce qui confirme que la propagation du contexte de trace fonctionne de bout en bout. Ce résultat est illustré en figure 19 en [annexe](#), qui présente une trace distribuée complète couvrant la chaîne front -> BS -> CS dans une seule vue.

Ce résultat valide le scénario 2, qui constituait le critère de validation le plus important de l'étude. La chaîne de corrélation ne commence plus à l'entrée de l'API backend, mais dès l'action utilisateur dans le navigateur. Il est désormais possible, à partir d'une seule vue dans Aspire, de reconstituer le parcours complet d'une interaction : l'ouverture de la popup, le déclenchement de la recherche, l'appel réseau vers le BS, les traitements distribués entre microservices, puis le retour des résultats au frontend.

Cette corrélation répond directement à la problématique posée en section 3.3 et à la limite principale identifiée dans l'analyse de l'existant en section 4.4 : l'absence de lien entre les actions utilisateur et les traitements backend.

9.3 Vérification de la confidentialité

L'inspection des attributs des spans produits dans Aspire confirme qu'aucune donnée personnelle ou sensible n'est remontée dans la télémétrie. Les valeurs de code postal et de nom de ville saisies par l'utilisateur lors de la recherche n'apparaissent dans aucun attribut des spans manuels ni des spans automatiques. Seuls des attributs fonctionnels agrégés et non nominatifs sont présents, comme le nombre de communes retournées par l'API ou le fait qu'une sélection ait été effectuée.

Ce résultat valide le scénario 3 et confirme que l'instrumentation respecte les contraintes de confidentialité définies en section 2.5. Le choix de ne porter dans les spans que des informations fonctionnelles non nominatives, tel que décrit en section 8.3.6, se révèle cohérent avec les bonnes pratiques du projet en matière de protection des données.

9.4 Analyse des performances

La comparaison des temps de réponse de l'appel GET /api/v1/communes, mesurés dans l'onglet Réseau des outils de développement du navigateur, ne révèle pas de dégradation perceptible entre une exécution sans instrumentation et une exécution avec instrumentation active. Les temps de réponse restent du même ordre de grandeur dans les deux cas.

Ce résultat s'explique par la conception non bloquante de l'instrumentation. Les spans sont créés et exportés de manière asynchrone, sans interférer avec le traitement de la requête elle-même. L'export des traces vers le proxy Vite se fait en arrière-plan, et le sérialiseur protobuf utilisé est léger en termes de consommation de ressources.

Le scénario 4 est donc satisfait : l'ajout de l'instrumentation ne dégrade pas l'expérience utilisateur sur le parcours instrumenté.

9.5 Limites de la solution

Malgré les résultats positifs obtenus, la solution mise en œuvre présente plusieurs limites qu'il convient de mentionner.

La première limite concerne le périmètre instrumenté. Seul un parcours utilisateur pilote a été instrumenté dans le cadre de cette étude. L'extension de l'instrumentation à l'ensemble des parcours de l'application nécessiterait un travail supplémentaire de définition des points d'observation pertinents et de maintenance de l'instrumentation manuelle dans le temps.

La deuxième limite tient à la dépendance au proxy Vite pour l'export des données. Ce mécanisme est adapté à l'environnement de développement, mais il ne serait pas directement transposable en production. Un déploiement en production nécessiterait de mettre en place un collecteur OpenTelemetry dédié, capable de recevoir les données depuis le navigateur et de les relayer vers un backend d'observabilité.

La troisième limite concerne la maturité de certains composants. Comme mentionné en section 6.2, certaines bibliothèques d'instrumentation automatique pour le navigateur restent à un stade expérimental dans l'écosystème OpenTelemetry. Cette situation peut évoluer avec les versions futures du SDK, mais elle invite à une vigilance particulière dans la maintenance à long terme.

Enfin, la solution actuelle ne couvre pas l'ensemble des signaux possibles de l'observabilité frontend. Les logs côté navigateur, les indicateurs Web Vitals et certaines métriques de performance perçue n'ont pas été intégrés dans cette première mise en œuvre. Ces dimensions constituent des pistes d'amélioration pertinentes pour enrichir l'observabilité frontend.

10 Conclusion et perspectives

10.1 Synthèses des résultats

L'objectif de cette étude était d'améliorer l'observabilité globale d'une application web en intégrant une télémétrie frontend, afin de permettre une corrélation efficace entre les interactions utilisateur et les traitements backend. Pour y répondre, l'étude s'est appuyée sur une analyse de l'existant, un état de l'art sur les concepts d'observabilité applicative, une étude comparative de plusieurs

approches d'instrumentation, puis une mise en œuvre concrète validée sur un parcours utilisateur pilote.

L'analyse de l'existant a mis en évidence une base d'observabilité backend déjà structurée, reposant sur OpenTelemetry, l'export OTLP et la propagation du contexte de trace entre microservices. Cette base constituait un socle solide, mais elle présentait une limite claire : l'absence totale de visibilité sur ce qui se passe dans le navigateur avant que la requête n'atteigne le backend. Les traces commençaient trop tard par rapport au parcours réel de l'utilisateur.

L'état de l'art a confirmé que l'observabilité frontend constitue un complément essentiel dans les architectures distribuées modernes, et que la corrélation entre le navigateur et les services backend représente un enjeu central pour disposer d'une vision de bout en bout du comportement d'une application.

La solution retenue, une approche hybride OpenTelemetry combinant instrumentation automatique des appels réseau et instrumentation manuelle des étapes fonctionnelles, a été mise en œuvre sur le parcours de recherche de lieu de garage dans le frontend FIA-Production_Vehicule. Cette mise en œuvre a nécessité de résoudre plusieurs contraintes techniques propres à l'environnement navigateur, notamment l'incompatibilité avec le protocole gRPC, le format d'export imposé par Aspire (protobuf uniquement), la gestion des politiques CORS via un proxy Vite et la restriction de la propagation du traceparent aux seules requêtes internes.

Les résultats obtenus valident les quatre objectifs définis en début d'étude. Le premier objectif est atteint : les spans manuels et automatiques produits par le frontend apparaissent dans le dashboard Aspire, aux côtés des traces backend déjà présentes. Le deuxième objectif, le plus important, est également validé : les spans frontend et backend partagent le même `traceId`, ce qui démontre que la corrélation fonctionne de bout en bout, depuis l'action utilisateur dans le navigateur jusqu'aux traitements distribués dans les microservices. Le troisième objectif est satisfait : aucune donnée personnelle ou sensible n'est remontée dans les attributs des spans. Le quatrième objectif est confirmé : l'ajout de l'instrumentation n'introduit pas de dégradation perceptible sur le parcours instrumenté.

L'étude démontre donc qu'il est possible, dans le cadre d'un projet existant et sans modification de l'architecture backend, d'étendre la chaîne d'observabilité jusqu'au navigateur et d'obtenir une vision continue du comportement de l'application, du clic utilisateur jusqu'aux traitements serveur.

10.2 Améliorations possibles

Plusieurs améliorations peuvent être envisagées pour renforcer la solution mise en œuvre.

La première amélioration concerne l'extension du périmètre instrumenté. Dans le cadre de cette étude, seul un parcours utilisateur pilote a été instrumenté. L'ajout de spans manuels sur d'autres parcours fonctionnels de l'application permettrait d'élargir la couverture de l'observabilité frontend et d'offrir une vision plus complète des interactions utilisateur. Le pattern d'instrumentation étant identique pour chaque point d'observation, cette extension ne nécessiterait pas de modification de l'architecture, mais un travail de définition des étapes pertinentes à tracer dans chaque parcours.

La deuxième amélioration porte sur l'intégration des indicateurs Web Vitals dans la chaîne de télémétrie. Comme évoqué dans l'état de l'art en section 5.6.2, les Core Web Vitals (LCP, INP, CLS, TTFB) permettent de quantifier objectivement la performance perçue par l'utilisateur. Leur collecte

via le SDK OpenTelemetry et leur remontée dans le dashboard permettraient de relier des indicateurs d'expérience utilisateur aux traces techniques déjà disponibles, enrichissant ainsi la lecture fonctionnelle de l'observabilité.

La troisième amélioration concerne la collecte des erreurs côté navigateur. L'instrumentation actuelle ne couvre pas de manière systématique les exceptions JavaScript non gérées, les échecs de chargement de ressources ou les erreurs réseau. L'ajout d'une instrumentation dédiée à ces événements, par exemple via le package `@opentelemetry/instrumentation-web-vitals` ou une capture manuelle des erreurs globales, permettrait de compléter la couverture de l'observabilité frontend.

Enfin, une quatrième amélioration pourrait porter sur l'exploitation des métriques frontend déjà mises en place. Les quatre métriques définies dans `otel.ts` (`fia.page_views`, `fia.user_actions`, `fia.errors`, `fia.api_call_duration`) constituent une base exploitable, mais elles ne sont pas encore utilisées de manière systématique dans les composants de l'application. L'intégration progressive de ces compteurs dans les parcours fonctionnels permettrait de disposer de données quantitatives plus riches sur l'activité côté navigateur.

10.3 Perspectives

Au-delà des améliorations directes de la solution, cette étude ouvre plusieurs perspectives pour le projet et pour la réflexion sur l'observabilité dans les applications web.

La première perspective concerne le passage en production. La solution actuelle repose sur un proxy Vite adapté à l'environnement de développement, qui ne serait pas directement transposable en production. Pour industrialiser l'observabilité frontend, il serait nécessaire de mettre en place un collecteur OpenTelemetry dédié, capable de recevoir les données depuis le navigateur via HTTP/protobuf et de les relayer vers un backend d'observabilité tel que Grafana Tempo, Jaeger ou une plateforme compatible OTLP. Cette architecture permettrait de conserver la même logique de collecte tout en s'adaptant aux contraintes d'un environnement de production.

La deuxième perspective porte sur la collaboration entre les équipes frontend et backend. L'un des apports de cette étude est de montrer qu'une vision unifiée du système, du navigateur jusqu'aux microservices, facilite la compréhension partagée du comportement de l'application. En rendant visibles les actions utilisateur dans les mêmes outils que les traces backend, l'observabilité frontend peut contribuer à améliorer les échanges entre les équipes, à accélérer le diagnostic d'incidents et à réduire les incompréhensions liées au cloisonnement des périmètres techniques.

La troisième perspective concerne l'évolution de l'écosystème OpenTelemetry pour le navigateur. Comme mentionné en section 6.2, certains composants du SDK JavaScript restent à un stade expérimental. L'évolution de ces bibliothèques vers une maturité stable pourrait permettre, à terme, d'enrichir l'instrumentation automatique avec des signaux plus fins, comme les interactions utilisateur détaillées ou les métriques de rendu, sans augmenter significativement l'effort de maintenance.

Enfin, cette étude s'inscrit dans une réflexion plus large sur la place de l'observabilité dans le cycle de développement logiciel. L'intégration de la télémétrie dès la conception des composants, plutôt qu'en complément a posteriori, pourrait devenir une pratique standard dans les projets d'applications web distribuées. L'approche hybride validée dans cette étude, combinant automatisation technique et instrumentation fonctionnelle ciblée, constitue un modèle réutilisable

et progressif, adapté à des projets qui souhaitent améliorer leur observabilité sans remettre en cause leur architecture existante.

11 Annexes

Figure 1 : Extrait de configuration OpenTelemetry dans le fichier Program.cs d'un microservice backend.....	35
Figure 2 : Suite de la configuration OpenTelemetry dans le fichier Program.cs	35
Figure 3 : Exemple de métriques visualisées dans le Dashboard .NET Aspire	36
Figure 4 : Exemple de traces distribuées visualisées dans le Dashboard .NET Aspire	36
Figure 5 : Complémentarité entre monitoring et observabilité.....	36
Figure 6 : les trois piliers de l'observabilité	37
Figure 7 : Architecture simplifiée de collecte et d'exploitation des données avec OpenTelemetry	37
Figure 8 : Corrélation entre le frontend instrumenté et les services backend	37
Figure 9 : Architecture de la solution proposée : chaîne de télémétrie frontend / backend	38
Figure 10 : Capture du formulaire et de la popup lieu de garage part1	38
Figure 11 : Capture du formulaire et de la popup lieu de garage part2	39
Figure 12 : Capture du fichier otel.ts : configuration du TracerProvider, exporteur protobuf, instrumentations automatiques	39
Figure 13 : Capture du fichier main.tsx : import de otel.ts en première ligne.....	40
Figure 14 : Capture de la configuration des métriques dans otel.ts : MeterProvider et les 4 métriques définies.....	40
Figure 15 : Capture du code d'instrumentation manuelle dans FormUsageVehicule.tsx : span garage.popup.open.....	41
Figure 16 : Capture du code d'instrumentation manuelle dans PopupLieuGarageContent.tsx	41
Figure 17: Capture de la trace FIA dans l'onglet Traces d'Aspire : spans manuels visibles.....	42
Figure 18: Capture des métriques FIA dans l'onglet Metrics d'Aspire : compteurs et histogramme ...	42
Figure 19 : Capture de la trace distribuée complète front -> BS -> CS dans Aspire : même traceId partagé entre les services	43

```

builder.Services.AddOpenTelemetry()
    .ConfigureResource(resource =>
    {
        resource.AddService(builder.Configuration["OTLP_SERVICE_NAME"]!);
        resource.AddAttributes(
            new Dictionary<string, object>
            {
                ["matmut.app.version"] = version,
                ["matmut.dotnet.processid"] = Environment.ProcessId,
                ["matmut.dotnet.framework"] = RuntimeInformation.FrameworkDescription,
            });
        resource.AddTelemetrySdk();
    })
    .WithMetrics(metrics =>
    {
        metrics.AddAspNetCoreInstrumentation()
            .AddHttpClientInstrumentation()
            .AddRuntimeInstrumentation()
            .AddProcessInstrumentation();
    })
    .WithTracing(tracing =>
    {
        tracing.AddSource(".*")
            .AddAspNetCoreInstrumentation(options =>

```

Figure 1 : Extrait de configuration OpenTelemetry dans le fichier Program.cs d'un microservice backend

```

        tracing.AddSource(".*")
            .AddAspNetCoreInstrumentation(options =>
            {
                options.Filter = contexte =>
                {
                    !string.Equals(contexte.Request.Path.Value, "/health", StringComparison.Ordinal)
                    && !string.Equals(contexte.Request.Path.Value, "/favicon.ico", StringComparison.Ordinal)
                    && !(contexte.Request.Path.Value?.StartsWith("/metrics", StringComparison.Ordinal) ?? false)
                    && !(contexte.Request.Path.Value?.StartsWith("/swagger", StringComparison.Ordinal) ?? false)
                }

                options.EnrichWithHttpRequest = (activity, request) =>
                {
                    if (request.Headers.TryGetValue("x-api-key", out var value))
                    {
                        var text = value.ToString();
                        var length = text.Length;
                        var value2 = string.Concat(text.AsSpan(0, length / 4), new string('*', length / 2), tex
                        activity.AddTag("matmut.x-api-key", value2);
                    }
                };
            })
            .AddHttpClientInstrumentation()
            .AddEntityFrameworkCoreInstrumentation(options =>
            {
                options.SetDbStatementForText = true;
            });
    })
    .UseOtlpExporter();

```

Figure 2 : Suite de la configuration OpenTelemetry dans le fichier Program.cs

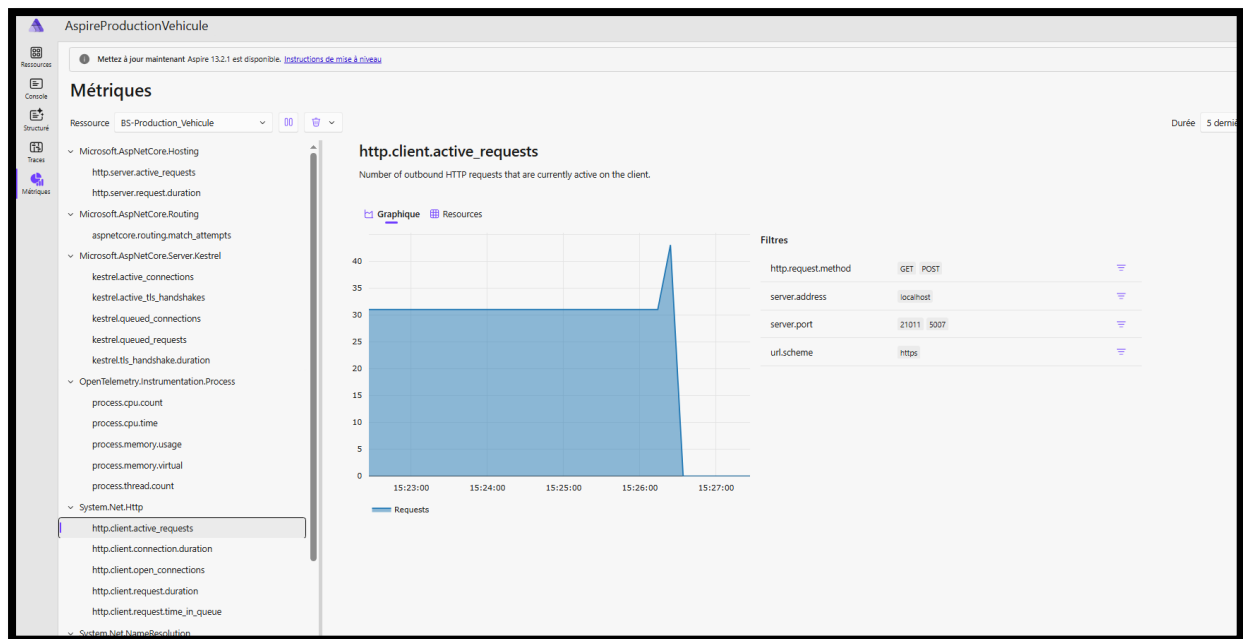


Figure 3 : Exemple de métriques visualisées dans le Dashboard .NET Aspire



Figure 4 : Exemple de traces distribuées visualisées dans le Dashboard .NET Aspire

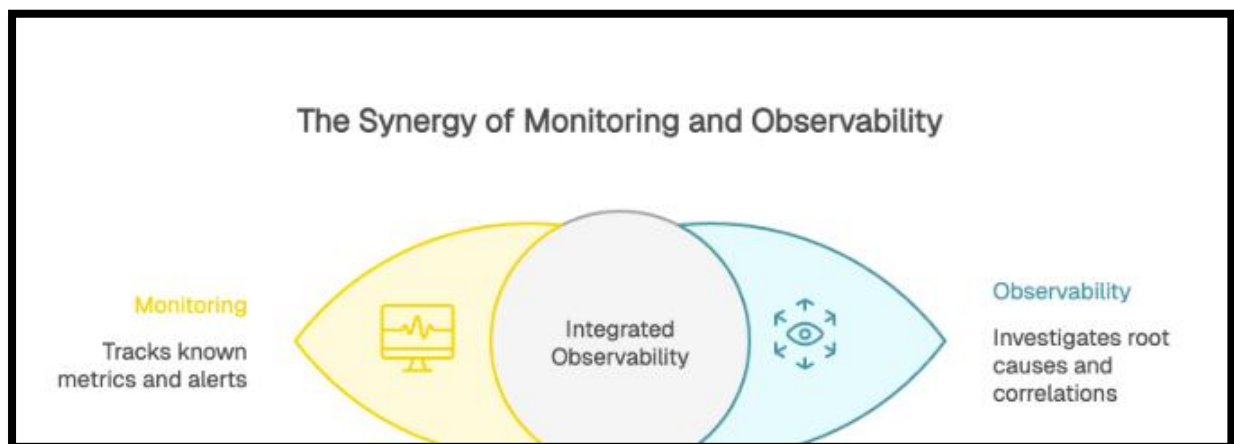


Figure 5 : Complémentarité entre monitoring et observabilité

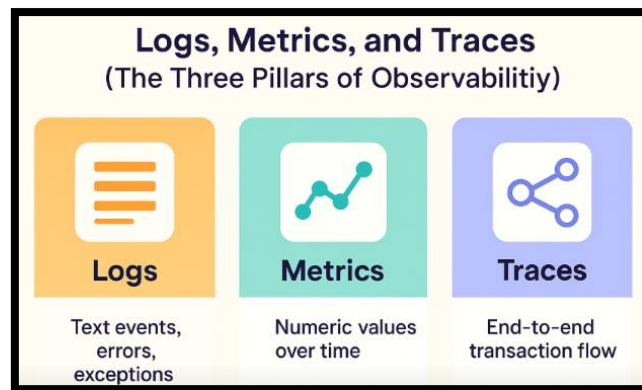


Figure 6 : les trois piliers de l'observabilité

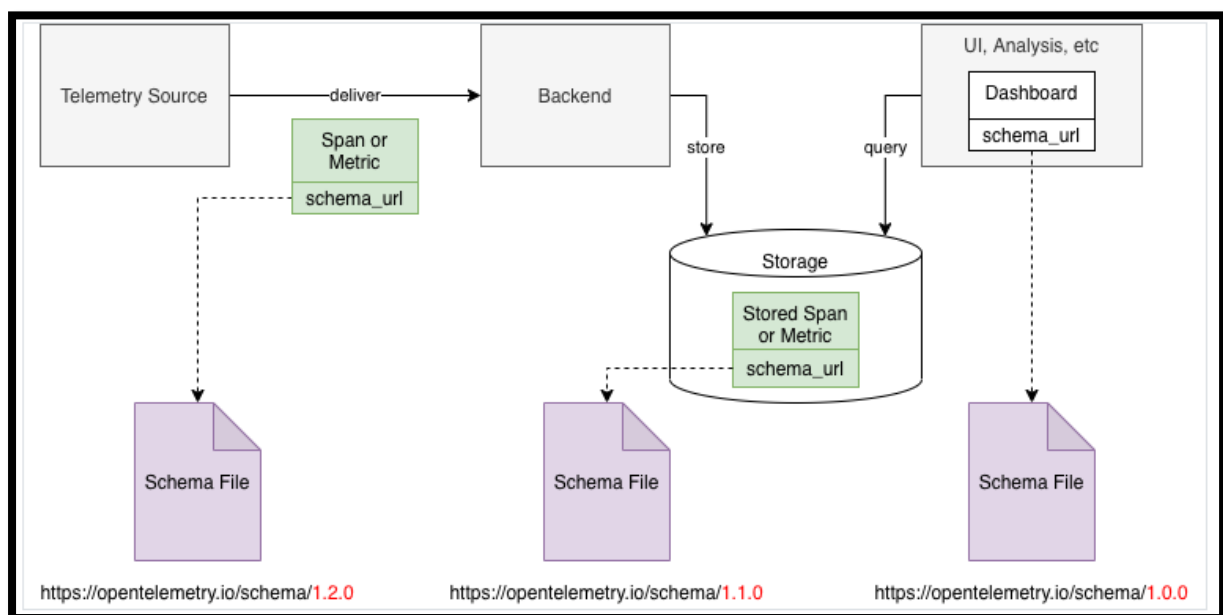


Figure 7 : Architecture simplifiée de collecte et d'exploitation des données avec OpenTelemetry

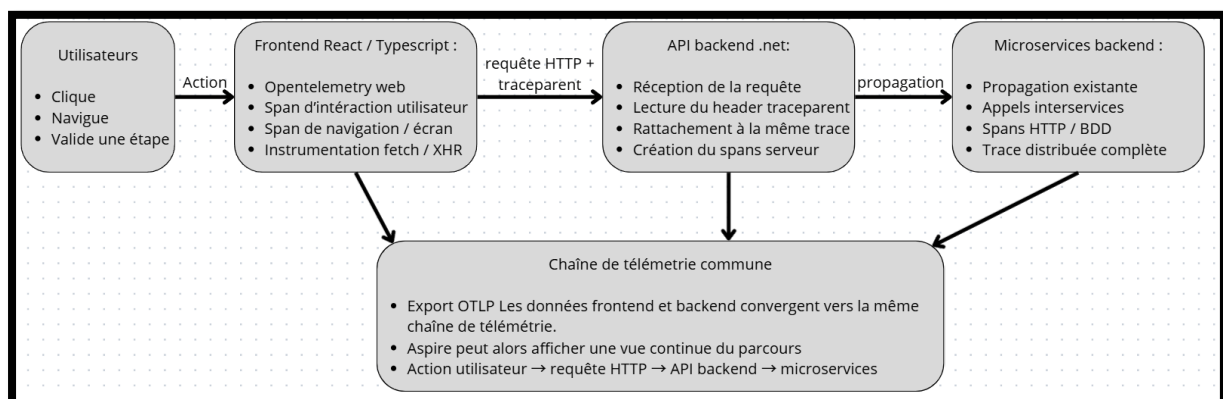


Figure 8 : Corrélation entre le frontend instrumenté et les services backend

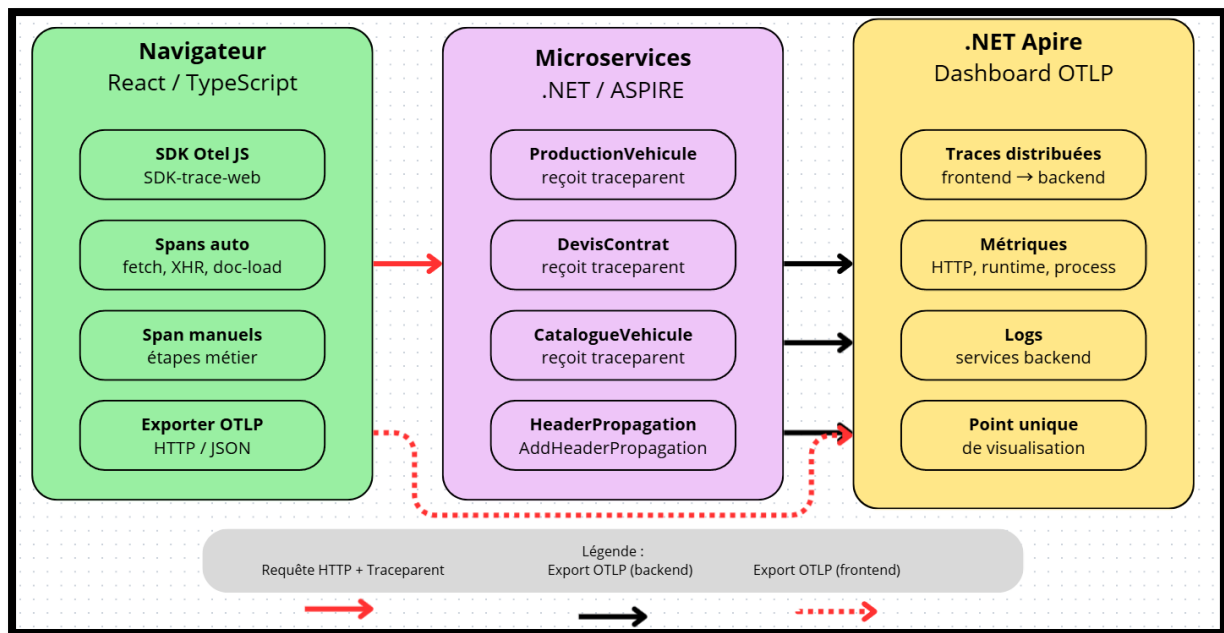


Figure 9 : Architecture de la solution proposée : chaîne de télémétrie frontend / backend

The screenshot shows a web form titled "Rechercher le lieu de garage". It includes a search bar with two input fields: "Code postal" and "Ville". A magnifying glass icon is positioned to the right of the "Ville" field. Below the search bar, there are two buttons: "Effacer l'écran" (with a trash icon) and "Reporter le lieu de garage" (with a location pin icon). The form is part of a larger application, as indicated by the "Fermer" button in the top right corner.

Figure 10 : Capture du formulaire et de la popup lieu de garage part1

Rechercher le lieu de garage

74000 Ville

Lieu-dit / Commune	Code Postal
ANNECY	74000

Effacer l'écran Reporter le lieu de garage

Figure 11 : Capture du formulaire et de la popup lieu de garage part2

```
import { ZoneContextManager } from '@opentelemetry/context-zone'
import { W3CTraceContextPropagator } from '@opentelemetry/core'
import { OTLPTraceExporter } from '@opentelemetry/exporter-trace-otlp-proto'
import { DocumentLoadInstrumentation } from '@opentelemetry/instrumentation-document-load'
import { FetchInstrumentation } from '@opentelemetry/instrumentation-fetch'
import { XMLHttpRequestInstrumentation } from '@opentelemetry/instrumentation-xml-http-request'
import { resourceFromAttributes } from '@opentelemetry/resources'
import { BatchSpanProcessor, WebTracerProvider } from '@opentelemetry/sdk-trace-web'
import { ATTR_SERVICE_NAME } from '@opentelemetry/semantic-conventions'

// -----
// 1. Ressource décrivant le service (nom affiché dans le dashboard Aspire)
// -----
const resource = resourceFromAttributes({
  [ATTR_SERVICE_NAME]: 'fia-production-vehicule',
})
const otlpExporter = new OTLPTraceExporter({
  url: '/otlp/v1/traces',
})
const provider = new WebTracerProvider({
  resource,
  spanProcessors: [new BatchSpanProcessor(otlpExporter)],
})
provider.register({
  contextManager: new ZoneContextManager(),
  propagator: new W3CTraceContextPropagator(),
})
const fetchInstrumentation = new FetchInstrumentation({
  propagateTraceHeaderCorsUrls: [/^https?:\/\/localhost(:\d+)?/],
})
const xhrInstrumentation = new XMLHttpRequestInstrumentation({
  propagateTraceHeaderCorsUrls: [/^https?:\/\/localhost(:\d+)?/],
})
const documentLoadInstrumentation = new DocumentLoadInstrumentation()
fetchInstrumentation.setTracerProvider(provider)
xhrInstrumentation.setTracerProvider(provider)
documentLoadInstrumentation.setTracerProvider(provider)
export const tracer = provider.getTracer('fia-production-vehicule')
```

Figure 12 : Capture du fichier otel.ts : configuration du TracerProvider, exporteur protobuf, instrumentations automatiques

```
100, 21 hours ago | 4 authors (BEKHAJIK Ayoub (Ext) and Others)  
// Initialisation OpenTelemetry – doit rester le tout premier import  
import './otel'  
  
import { StrictMode } from 'react'  
import ReactDOM from 'react-dom/client'  
import App from './App'  
import './config/i18n'  
import './styles/globals.css'  
  
ReactDOM.createRoot(document.getElementById('root')!).render(  
  <StrictMode>  
    <App />  
  </StrictMode>,  
)
```

Figure 13 : Capture du fichier main.tsx : import de otel.ts en première ligne

```
export const metrics = {  
  /** Compteur de pages vues – attribut recommandé : page.name */  
  pageViews: meter.createCounter('fia.page_views', {  
    description: 'Nombre de pages vues par l\'utilisateur',  
    unit: '{views}',  
  })),  
  
  /** Compteur d'actions utilisateur – attribut recommandé : action.type */  
  userActions: meter.createCounter('fia.user_actions', {  
    description: 'Nombre d\'actions utilisateur (clics, soumissions, etc.)',  
    unit: '{actions}',  
  })),  
  
  /** Compteur d'erreurs frontend – attribut recommandé : error.type */  
  errors: meter.createCounter('fia.errors', {  
    description: 'Nombre d\'erreurs côté frontend',  
    unit: '{errors}',  
  })),  
  
  /** Histogramme de durée des appels API – attribut recommandé : api.endpoint */  
  apiCallDuration: meter.createHistogram('fia.api_call_duration', {  
    description: 'Durée des appels API depuis le frontend',  
    unit: 'ms',  
  })),  
}
```

Figure 14 : Capture de la configuration des métriques dans otel.ts : MeterProvider et les 4 métriques définies

```

<Button
  onClick={() => {
    // Span OpenTelemetry : ouverture de la popup de recherche de lieu de garage
    tracer.startActiveSpan('garage.popup.open', (span) => {
      span.end()
    })
    setIsModalOpen(true)
  }}
  variant="lite"
  iconOnly
  aria-label="Rechercher le lieu de garage dans la base postale"
  type="button"
>
<Icon name="icone-maison-question" />

```

Figure 15 : Capture du code d'instrumentation manuelle dans FormUsageVehicule.tsx : span garage.popup.open

```

const handleSearch = async () => {
  if (await trigger(['cpGarage', 'communeGarage'])) {
    // Span OpenTelemetry : déclenchement de la recherche dans la popup
    tracer.startActiveSpan('garage.search.triggered', (span) => {
      span.end()
    })
    resetField('selectedLocation')
    fetchCommunes()
  }
}

const handleReport = async () => {
  if (await trigger()) {
    const selected = communes!.find(res => `${res.codePostal}-${res.nom}` === selectedLocation)!

    // Span OpenTelemetry : validation du lieu de garage – aucune donnée personnelle
    tracer.startActiveSpan('garage.location.confirmed', (span) => {
      span.end()
    })

    onReport(selected)
  }
}

```

Figure 16 : Capture du code d'instrumentation manuelle dans PopuLieuGarageContent.tsx

Traces				
Ressource (Tout)		00		Q Filtrer... Type (Tout)
Horodatage	Nom	Étendues		Durée
14:48:01,677	fia-production-vehicule: HTTP GET 367e8ae	fia-production-vehicule (1)	BS-Production_Vehicule (3)	CS-Devis_Contrats_Vehicule (2) +1 0.39s
14:48:01,677	fia-production-vehicule: HTTP GET c2ab6d1	fia-production-vehicule (1)	BS-Production_Vehicule (3)	CS-Devis_Contrats_Vehicule (2) +1 0.38s
14:48:01,678	fia-production-vehicule: HTTP GET b016163	fia-production-vehicule (1)		74ms
14:48:01,678	fia-production-vehicule: HTTP GET d23a183	fia-production-vehicule (1)		70ms
14:48:01,993	BS-Production_Vehicule: TLS server handshake bfe2944	BS-Production_Vehicule (1)		0.03ms
14:48:01,995	BS-Production_Vehicule: OPTIONS a35ac6f	BS-Production_Vehicule (1)		1.04ms
14:48:01,996	BS-Production_Vehicule: OPTIONS ef196d8	BS-Production_Vehicule (1)		0.83ms
14:48:02,002	BS-Production_Vehicule: HTTP connection_setup localhost:5007 2d59642	BS-Production_Vehicule (4)		17.25ms
14:48:02,002	BS-Production_Vehicule: HTTP connection_setup localhost:5007 12b62f1	BS-Production_Vehicule (4)		17.36ms
14:48:02,009	CS-Devis_Contrats_Vehicule: TLS server handshake e6ff112	CS-Devis_Contrats_Vehicule (1)		0.05ms
14:48:02,009	CS-Devis_Contrats_Vehicule: TLS server handshake 675240d	CS-Devis_Contrats_Vehicule (1)		0.02ms
14:48:02,297	BS-Production_Vehicule: TLS server handshake 4ae83c2	BS-Production_Vehicule (1)		0.02ms
14:48:05,197	fia-production-vehicule: garage.popup.open 01a2138	fia-production-vehicule (1)		0.1ms
14:48:08,825	fia-production-vehicule: garage.search.triggered 04db69b	fia-production-vehicule (1)		0µs
14:48:08,830	fia-production-vehicule: HTTP GET db3dd9d	fia-production-vehicule (1)		49ms
14:48:08,879	fia-production-vehicule: garage.search.results 5874b8b	fia-production-vehicule (1)		0µs
14:48:10,932	fia-production-vehicule: garage.location.confirmed 77e7512	fia-production-vehicule (1)		0.1ms

Figure 17: Capture de la trace FIA dans l'onglet Traces d'Aspire : spans manuels visibles

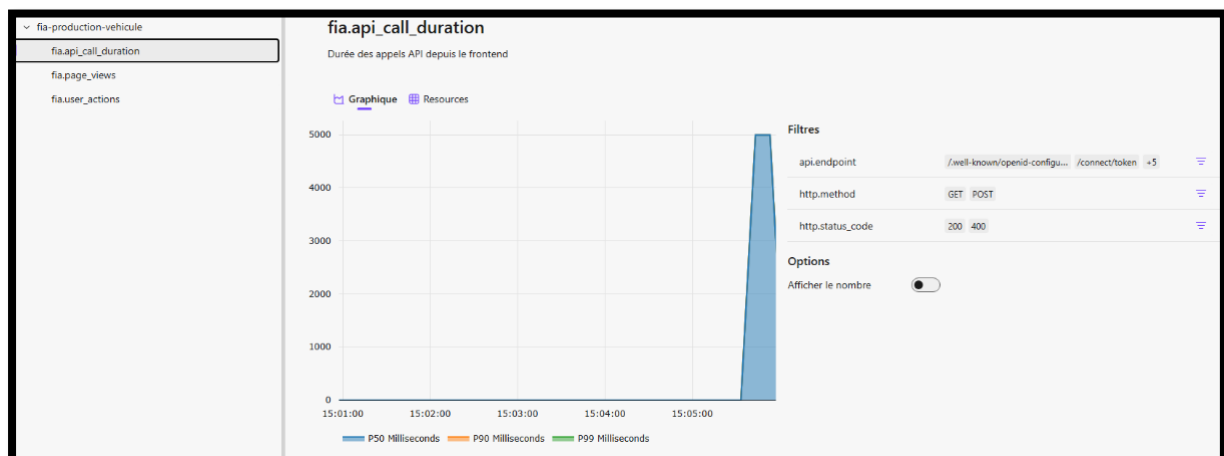


Figure 18: Capture des métriques FIA dans l'onglet Metrics d'Aspire : compteurs et histogramme

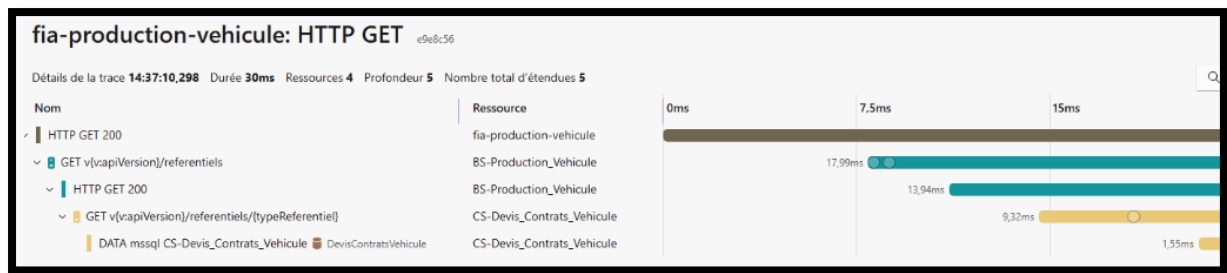


Figure 19 : Capture de la trace distribuée complète front -> BS -> CS dans Aspire : même traceId partagée entre les services

12 Bibliographie

- [1] Cloud Native Computing Foundation. OpenTelemetry. Disponible sur : <https://www.cncf.io/projects/opentelemetry/>
- [2] Datadog. OpenTelemetry Integration. Disponible sur : <https://www.datadoghq.com/dg/apm/opentelemetry/>
- [3] Microsoft. .NET Aspire dashboard overview. Disponible sur : <https://learn.microsoft.com/fr-fr/dotnet/aspire/fundamentals/dashboard/overview>
- [4] OpenTelemetry. Documentation officielle. Disponible sur : <https://opentelemetry.io/docs/>
- [5] OpenTelemetry. Getting Started with OpenTelemetry in the Browser. Disponible sur : <https://opentelemetry.io/docs/languages/js/getting-started/browser/>
- [6] OpenTelemetry. Schema Specification. Disponible sur : <https://opentelemetry.io/docs/specs/otel/schemas/>
- [7] Red Hat. What is Observability? Disponible sur : <https://www.redhat.com/fr/topics/devops/what-is-observability>